

Documentación del Proyecto

Gestor de Horarios de Maestros

1. Introducción

El proyecto **Gestor de Horarios de Maestros** es una aplicación diseñada para facilitar la organización y administración de los horarios de los docentes dentro de una institución educativa.

El sistema permite registrar maestros, asignar materias y organizar los horarios de clases de forma estructurada, evitando conflictos de horario y mejorando la planificación académica.

Este proyecto forma parte de la asignatura **Proyecto y Gobierno de TI**, donde se aplican conocimientos de desarrollo de software, gestión de información y organización de datos.

2. Objetivos del Proyecto

Objetivo general

Desarrollar un sistema que permita **gestionar y organizar los horarios de los maestros** de manera eficiente.

Objetivos específicos

- Registrar información de los maestros.
 - Administrar materias o asignaturas.
 - Crear horarios de clases.
 - Evitar conflictos de horario.
 - Facilitar la consulta de horarios.
-

3. Justificación

En muchas instituciones educativas, la creación de horarios se realiza de forma manual, lo que puede generar problemas como:

- Conflictos entre clases.
- Duplicación de horarios.
- Dificultad para consultar la información.

Este sistema busca **automatizar la organización de los horarios**, permitiendo una mejor gestión académica y reduciendo errores humanos.

Los sistemas de gestión académica ayudan a optimizar la planificación escolar y el uso de recursos educativos. ([REDI](#))

4. Alcance del Sistema

El sistema permite:

- Registrar maestros.
- Registrar materias.
- Crear horarios.
- Consultar horarios de docentes.

No incluye:

- Gestión de estudiantes.
- Sistema de calificaciones.
- Plataforma educativa.

5. Tecnologías Utilizadas

Dependiendo del repositorio, normalmente incluye tecnologías como:

- Lenguaje de programación (por ejemplo Java, Python o JavaScript).
- Base de datos.
- Herramientas de desarrollo.
- Control de versiones con Git.

El proyecto se encuentra alojado en un repositorio de control de versiones para facilitar el seguimiento y organización del código.

6. Arquitectura del Sistema

El sistema puede dividirse en varios componentes:

1. Módulo de maestros

Permite registrar y gestionar la información de los docentes.

Funciones:

- Crear maestro
- Editar maestro
- Consultar maestros

2. Módulo de materias

Permite registrar las asignaturas disponibles.

Funciones:

- Registrar materias
 - Consultar materias
-

3. Módulo de horarios

Se encarga de asignar los horarios de clase.

Funciones:

- Crear horario
 - Modificar horario
 - Consultar horario por maestro
-

7. Requerimientos del Sistema

Requerimientos funcionales

El sistema debe permitir:

1. Registrar maestros.
 2. Registrar materias.
 3. Crear horarios.
 4. Consultar horarios.
 5. Modificar horarios existentes.
-

Requerimientos no funcionales

- El sistema debe ser fácil de usar.
 - Debe mantener organizada la información.
 - Debe evitar conflictos de horarios.
-

8. Casos de Uso

Crear horario

Actor: Administrador

Proceso:

1. El administrador selecciona el maestro.

2. Selecciona la materia.
3. Define el día.
4. Define la hora.
5. El sistema guarda el horario.

Resultado:

El horario queda registrado en el sistema.

9. Estructura del Proyecto

El proyecto es una aplicación de escritorio desarrollada en C# con Windows Forms (.NET). Toda la lógica y la interfaz se encuentran en una única solución de Visual Studio. La estructura del repositorio es la siguiente:

Proyecto-Final---Proyecto-y-Gobierno-de-TI/

|— Gestor de Horarios de Maestros/ ← Carpeta principal del proyecto

| |— Gestor de Horarios de Maestros.csproj

| |— Gestor de Horarios de Maestros.slnx

| |— Program.cs

| |— Principal.cs

| |— FormAgregar.cs

| |— FormAsignar.cs

| |— FormBuscar.cs

| |— FormImprimir.cs

| |— FormModificar.cs

| |— ConfigConexion.cs

| |— ValidadorHorario.cs

| |— app.config

| |— Properties/

| |— Resources/

| |— bin/ y obj/ ← Carpetas generadas al compilar

|— README.md

|— Instrucciones de Instalacion.txt

10. Instalación del Proyecto

Requisitos previos

- Sistema operativo: Windows 10 o superior
- Visual Studio (2022 recomendado)
- MySQL Server instalado y en ejecución
- .NET Framework compatible con el proyecto

Pasos para instalar y ejecutar el sistema:

1. Descargar o clonar el repositorio
Descarga el proyecto como ZIP o clónalo con Git.
2. Abrir la solución en Visual Studio Abre la carpeta Gestor de Horarios de Maestros y carga el archivo Gestor de Horarios de Maestros.slnx.
3. Configurar la base de datos
 - Instala MySQL Server (si no lo tienes).
 - Crea la base de datos correspondiente al sistema (puedes usar los scripts o la estructura que se indica en la documentación del proyecto).
4. Configurar la conexión a la base de datos
Ejecuta la aplicación y, desde el menú Conexión de la interfaz, ingresa los datos de tu servidor MySQL (servidor, usuario, contraseña y nombre de la base de datos).
5. Ejecutar la aplicación
Presiona F5 en Visual Studio o ejecuta directamente el archivo .exe ubicado en la carpeta bin/Debug/net10.0-windows.

Una vez ejecutado, el sistema permitirá gestionar horarios de maestros, asignar materias, buscar, modificar, eliminar y visualizar los horarios en la tabla principal.

11. Conclusión

El **Gestor de Horarios de Maestros** permite organizar de forma eficiente la planificación académica dentro de una institución educativa.

La automatización del proceso de creación de horarios reduce errores humanos, mejora la organización de las clases y facilita el acceso a la información por parte de los administradores.

Este proyecto demuestra la aplicación de conceptos de desarrollo de software, gestión de datos y organización de sistemas informáticos.

En un proyecto de **Windows Forms en C#**, cada formulario normalmente se divide en **tres partes principales** para mantener el código organizado y fácil de mantener.

Te lo explico de forma clara para que puedas usarlo también en tu **documentación del proyecto**.

Partes de un Formulario en un Proyecto de Programación (Windows Forms)

En aplicaciones desarrolladas con **Windows Forms** dentro de **Microsoft Visual Studio**, cada formulario suele estar compuesto por **tres archivos principales** que separan las responsabilidades del programa.

1. Lógica del Formulario

Archivo: FormAsignar.cs

Esta parte contiene **la lógica de funcionamiento del formulario**.

Aquí se programan:

- Conexiones a la base de datos
- Procesamiento de datos
- Eventos de botones
- Validaciones
- Métodos del formulario

Ejemplo de acciones que ocurren aquí:

- Cargar maestros desde la base de datos.
- Cargar materias disponibles.
- Ejecutar la consulta SQL para asignar un maestro a una materia.
- Manejar errores.

Ejemplo del evento principal:

`btnGuardar_Click()`

Este método ejecuta la consulta SQL que asigna el maestro a la materia.

En términos de arquitectura del software, esta parte representa la **lógica de la aplicación**.

2. Diseño de la Interfaz

Archivo: FormAsignar.Designer.cs

Este archivo contiene **la definición visual del formulario**.

Aquí se crean y configuran todos los controles de la interfaz gráfica, como:

- Botones
- Etiquetas (Label)

- Listas desplegables (ComboBox)
- Cuadros de texto
- Tamaño del formulario
- Posición de los elementos

Ejemplo de controles definidos en este archivo:

Control	Función
cmbMaestros	Lista de maestros
cmbMaterias	Lista de materias
btnGuardar	Botón para realizar la asignación
lbl1	Texto "Seleccionar Maestro"
lbl2	Texto "Seleccionar Materia"

Este archivo **lo genera automáticamente Visual Studio**, por lo que normalmente **no se modifica manualmente**.

3. Recursos del Formulario

Archivo: FormAsignar.resx

Este archivo almacena **recursos utilizados por el formulario**.

Los recursos pueden incluir:

- Textos
- Imágenes
- Iconos
- Configuraciones de controles
- Datos de localización (idiomas)

El archivo está escrito en **formato XML** y permite separar los recursos del código fuente.

Ejemplo de información dentro del archivo:

- Tipo de recurso
- Versiones de bibliotecas
- Metadatos del formulario

Este archivo también suele ser **generado automáticamente por el entorno de desarrollo**.

Relación entre los tres archivos

Los tres archivos trabajan juntos para crear el formulario completo.

Archivo	Responsabilidad
FormAsignar.cs	Lógica del formulario
FormAsignar.Designer.cs	Diseño de la interfaz
FormAsignar.resx	Recursos del formulario

En conjunto forman **un único formulario dentro de la aplicación**.

Ventajas de esta estructura

Esta división permite:

- Mantener el código organizado.
 - Facilitar el mantenimiento del sistema.
 - Separar lógica y diseño.
 - Permitir que el diseñador visual genere código automáticamente.
 - Mejorar la escalabilidad del proyecto.
-

Conclusión:

Sí, en un proyecto de **Windows Forms** cada formulario se compone normalmente de **tres partes fundamentales**:

1. **Lógica del formulario** (.cs)
2. **Diseño de la interfaz** (.Designer.cs)
3. **Recursos del formulario** (.resx)

Esta estructura es una práctica estándar utilizada en aplicaciones desarrolladas en **.NET con interfaces gráficas**.

11. Esquema de la Base de Datos (Inferido del Código Fuente)

La aplicación **Gestor de Horarios de Maestros** no incluye dentro del repositorio el script oficial para la creación de la base de datos.

Por esta razón, el esquema de la base de datos fue **inferido a partir del análisis del código fuente**, específicamente de las consultas SQL utilizadas por el sistema.

Antes de ejecutar el programa, es necesario **crear manualmente las tablas y la vista utilizadas por la aplicación.**

11.1 Tablas Principales

El sistema utiliza principalmente dos tablas: **Maestros** y **Materias**.

Tabla: Maestros

Esta tabla almacena la información básica de los docentes registrados en el sistema.

```
CREATE TABLE Maestros (  
    IdMaestro INT PRIMARY KEY AUTO_INCREMENT,  
    Nombre VARCHAR(100) NOT NULL  
);
```

Descripción de campos

Campo	Tipo	Descripción
IdMaestro	INT	Identificador único del maestro
Nombre	VARCHAR(100)	Nombre completo del maestro

Tabla: Materias

Esta tabla almacena las materias o asignaturas disponibles y su relación con los maestros.

```
CREATE TABLE Materias (  
    IdMateria INT PRIMARY KEY AUTO_INCREMENT,  
    Nombre VARCHAR(100) NOT NULL,  
    IdMaestro INT NULL,  
    FOREIGN KEY (IdMaestro) REFERENCES Maestros(IdMaestro)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Descripción de campos

Campo	Tipo	Descripción
IdMateria	INT	Identificador único de la materia
Nombre	VARCHAR(100)	Nombre de la asignatura
IdMaestro	INT	Maestro asignado a la materia

Relación entre tablas

- Un **maestro puede impartir varias materias**.
 - Una **materia puede tener un maestro asignado**.
 - Si un maestro es eliminado, la materia **no se elimina**, sino que su campo IdMaestro se establece en **NULL**.
-

11.2 Vista Utilizada por la Aplicación

La pantalla principal de la aplicación carga los datos a través de una **vista de base de datos llamada HorariosView**.

Esta vista combina información de las tablas **Maestros** y **Materias** mediante un JOIN.

```
CREATE VIEW HorariosView AS
```

```
SELECT
```

```
    m.Nombre AS MaestroNombre,
```

```
    mat.IdMateria,
```

```
    mat.Nombre AS Nombre,
```

```
    'Lunes' AS DiasImparte,
```

```
    '08:00' AS Hora,
```

```
    '3' AS HD_Credito,
```

```
    '30' AS DiasMes,
```

```
    '90' AS TotalCredito,
```

```
    '25' AS Inscritos,
```

```
    'A101' AS Aula,
```

```
    '01' AS Seccion,
```

```
    '3' AS Credito
```

```
FROM Materias mat
```

```
LEFT JOIN Maestros m ON mat.IdMaestro = m.IdMaestro;
```

Esta vista permite mostrar los datos necesarios para el **horario de clases** dentro de la interfaz de la aplicación.

11.3 Columnas Utilizadas por la Vista

Las siguientes columnas son utilizadas directamente por el código de la aplicación:

Columna	Descripción
MaestroNombre	Nombre del maestro
IdMateria	Identificador de la materia
Nombre	Nombre de la materia
DiasImparte	Día en que se imparte la clase
Hora	Hora de la clase
HD_Credito	Horas por crédito
DiasMes	Días del mes
TotalCredito	Total de créditos
Inscritos	Número de estudiantes inscritos
Aula	Aula asignada
Seccion	Sección del curso
Credito	Créditos de la materia

11.4 Notas Importantes sobre la Base de Datos

Durante el análisis del código fuente se identificaron las siguientes características importantes:

- Validación de choques de horarios

El sistema **no realiza la validación de horarios en la base de datos**.

Esta validación se realiza en memoria dentro del código de la aplicación, específicamente en la clase:

ValidadorHorario.cs

- Asignación de maestro a materia

La asignación de un maestro a una materia se realiza mediante una consulta SQL similar a la siguiente:

```
UPDATE Materias
```

```
SET IdMaestro = @idM
```

```
WHERE IdMateria = @idMat;
```

- Ausencia de tabla de Horarios

El sistema **no incluye una tabla específica llamada Horarios**.

Toda la lógica de horarios parece estar basada en:

- La tabla **Materias**
- La tabla **Maestros**
- La vista **HorariosView**

Los valores relacionados con días, horas, aulas y créditos se generan dentro de la vista o dentro de la lógica del programa.

12. Validación de Horarios

En el sistema **Gestor de Horarios de Maestros**, la validación de conflictos de horarios **no se realiza en la base de datos**, sino directamente en la lógica del programa utilizando C#.

Esto permite detectar conflictos antes de guardar los datos en el sistema.

La validación se implementa en la clase:

ValidadorHorarios.cs

Lógica de Validación

El método principal utilizado es HayChoque(), que verifica si un nuevo horario se superpone con alguno existente.

```
public class ValidadorHorarios
{
    public static bool HayChoque(HorarioSimple nuevo,
                                List<HorarioSimple> existentes,
                                out string mensaje)
    {
        foreach (var h in existentes)
        {
            if (h.Maestro == nuevo.Maestro && h.Dia == nuevo.Dia)
            {
                bool seCruzan = nuevo.HoraInicio < h.HoraFin &&
                                nuevo.HoraFin > h.HoraInicio;

                if (seCruzan)
                {
```

```

        mensaje = $"El maestro {nuevo.Maestro} ya tiene clase el {h.Dia} de {h.HoraInicio}:00
a {h.HoraFin}:00.";

        return true;
    }
}

mensaje = "";
return false;
}
}

```

Funcionamiento del algoritmo

El proceso funciona de la siguiente manera:

1. El usuario introduce un nuevo horario.
2. El sistema obtiene la lista de horarios existentes.
3. Se compara el nuevo horario con cada horario existente.
4. Si coinciden el **maestro y el día**, se verifica si las horas se superponen.
5. Si existe superposición, el sistema bloquea el registro y muestra un mensaje de error.

Condición de choque de horarios

El conflicto ocurre cuando:

$\text{HoraInicioNuevo} < \text{HoraFinExistente}$

AND

$\text{HoraFinNuevo} > \text{HoraInicioExistente}$

Esto indica que los intervalos de tiempo se cruzan.

13. Herramientas para Diagramas Entidad-Relación (ER)

Para documentar la estructura de la base de datos se recomienda utilizar herramientas de diagramación. Algunas de las más utilizadas son:

Herramienta	Tipo	Ventaja
diagrams.net (draw.io)	Gratuita	Muy fácil de usar
dbdiagram.io	Gratuita	Genera diagramas automáticamente

Herramienta	Tipo	Ventaja
Lucidchart	Freemium	Herramienta profesional
Mermaid	Código	Ideal para documentación en GitHub
PlantUML	Código	Diagramas avanzados

La herramienta recomendada para este proyecto es **diagrams.net**, debido a su facilidad de uso y compatibilidad con múltiples formatos de exportación.

14. Esquema Detallado de la Base de Datos

La base de datos utilizada por el sistema se denomina:

GestorHorarios

El esquema está compuesto por **dos tablas principales y una vista**.

14.1 Tabla Maestros

Almacena la información de los docentes registrados.

```
CREATE TABLE Maestros (
    IdMaestro INT PRIMARY KEY AUTO_INCREMENT,
    Nombre VARCHAR(100) NOT NULL
);
```

Campo	Tipo	Descripción
IdMaestro	INT	Identificador del maestro
Nombre	VARCHAR	Nombre del docente

14.2 Tabla Materias

Contiene las materias impartidas y su relación con los maestros.

```
CREATE TABLE Materias (
    IdMateria INT PRIMARY KEY,
    IdMaestro INT,
    Nombre VARCHAR(100) NOT NULL,
    DiasImparte VARCHAR(50),
    Hora VARCHAR(20),
```

```

HD_Credito INT,
DiasMes INT,
TotalCredito INT,
Inscritos INT,
Aula VARCHAR(20),
Seccion VARCHAR(10),
Credito INT,
FOREIGN KEY (IdMaestro) REFERENCES Maestros(IdMaestro)
ON DELETE SET NULL ON UPDATE CASCADE
);

```

Campo	Tipo	Descripción
IdMateria	INT	Identificador de la materia
IdMaestro	INT	Maestro asignado
Nombre	VARCHAR	Nombre de la materia
DiasImparte	VARCHAR	Día de la clase
Hora	VARCHAR	Hora del horario
HD_Credito	INT	Horas por crédito
DiasMes	INT	Días del mes
TotalCredito	INT	Créditos totales
Inscritos	INT	Estudiantes inscritos
Aula	VARCHAR	Aula asignada
Seccion	VARCHAR	Sección
Credito	INT	Créditos académicos

14.3 Vista HorariosView

La interfaz principal de la aplicación consulta los datos mediante una vista que une la información de las tablas.

```
CREATE VIEW HorariosView AS
```

```
SELECT
```

```
    m.Nombre AS MaestroNombre,
```

mat.IdMateria,
mat.Nombre,
mat.DiasImparte,
mat.Hora,
mat.HD_Credito,
mat.DiasMes,
mat.TotalCredito,
mat.Inscritos,
mat.Aula,
mat.Seccion,
mat.Credito

FROM Materias mat

LEFT JOIN Maestros m ON mat.IdMaestro = m.IdMaestro;

Esta vista facilita la consulta de datos para mostrar el horario completo dentro de la aplicación.

15. Mejores Prácticas de Gobierno de TI Aplicadas

El proyecto aplica varias prácticas asociadas al **Gobierno de Tecnologías de Información**, que buscan mejorar el control, la gestión y la calidad de los sistemas.

Gestión de configuración

La conexión a la base de datos se configura mediante el archivo:

app.config

Esto permite modificar los parámetros sin alterar el código fuente.

Gestión de riesgos

La implementación del **validador de horarios** evita conflictos en la asignación de clases, reduciendo errores operativos.

Seguridad

Las consultas SQL utilizan **parámetros**, lo que protege el sistema contra ataques de **inyección SQL**.

Gestión de cambios

El sistema separa las operaciones en distintos formularios:

- Agregar
- Modificar
- Eliminar

Esto facilita el control y mantenimiento del sistema.

Documentación y trazabilidad

El uso de un repositorio en **GitHub** permite llevar control de versiones y seguimiento de los cambios realizados en el proyecto.

16. Limitaciones del Sistema

A pesar de su funcionalidad, el sistema presenta algunas limitaciones:

- El identificador IdMateria se introduce manualmente.
 - La validación de horarios solo ocurre al agregar materias.
 - No existe una tabla específica para horarios.
 - No se incluye exportación de datos a Excel o PDF.
-

17. Mejoras Futuras

Para mejorar el sistema se podrían implementar las siguientes funcionalidades:

- Convertir IdMateria en AUTO_INCREMENT.
 - Crear una tabla independiente llamada Horarios.
 - Añadir validación también al modificar horarios.
 - Implementar reportes en PDF o Excel.
 - Agregar autenticación de usuarios.
-

Conclusión

El sistema **Gestor de Horarios de Maestros** permite administrar de manera organizada los horarios académicos de una institución educativa.

El uso de validaciones en el código, una estructura simple de base de datos y buenas prácticas de desarrollo permiten crear una solución funcional y adaptable. Además, el proyecto demuestra la aplicación de principios básicos de **Gobierno de TI**, como gestión de riesgos, control de cambios y documentación del sistema.

la **documentación técnica clara** del formulario **FormAsignar** y del archivo **FormAsignar.resx**, como normalmente se incluye en la **documentación de un sistema o manual técnico de software**.

18. Documentación del Módulo de Asignación de Maestros

Archivo: FormAsignar.cs

El archivo **FormAsignar.cs** pertenece al módulo encargado de **asignar un maestro a una materia** dentro del sistema **Gestor de Horarios de Maestros**.

Este formulario permite al usuario seleccionar:

- Un **maestro**
- Una **materia**

y posteriormente realizar la **asignación entre ambos registros** dentro de la base de datos.

El formulario forma parte de la interfaz desarrollada con **Windows Forms en C#**, utilizando una conexión a base de datos **MySQL**.

18.1 Dependencias del Archivo

El archivo utiliza las siguientes bibliotecas del entorno .NET:

Biblioteca	Función
System	Funciones básicas del sistema
System.Data	Manejo de estructuras de datos
System.Windows.Forms	Interfaz gráfica
MySql.Data.MySqlClient	Conexión con MySQL
System.Configuration	Lectura de configuración desde app.config

Estas dependencias permiten que el formulario interactúe con la base de datos y muestre información al usuario.

18.2 Conexión a la Base de Datos

La conexión se obtiene desde el archivo de configuración **app.config** mediante la siguiente propiedad:

```
string                                connectionString                                =>
ConfigurationManager.ConnectionStrings["MiConexion"].ConnectionString;
```

Esto permite mantener la cadena de conexión **fuera del código fuente**, lo que facilita su modificación sin recompilar la aplicación.

18.3 Constructor del Formulario

```
public FormAsignar()
{
    InitializeComponent();
    CargarDatos();
}
```

El constructor realiza dos acciones principales:

1. Inicializa los componentes de la interfaz gráfica.
 2. Ejecuta el método CargarDatos() para llenar los controles del formulario.
-

18.4 Método CargarDatos()

Este método se encarga de **obtener la información de la base de datos** y cargarla en los controles ComboBox.

Operaciones realizadas

1. Se abre una conexión con la base de datos.
2. Se ejecuta una consulta SQL para obtener los maestros.
3. Se ejecuta otra consulta para obtener las materias.
4. Los resultados se almacenan en objetos DataTable.
5. Los datos se asignan a los controles del formulario.

Consulta para cargar maestros

```
SELECT IdMaestro, Nombre FROM Maestros
```

Estos datos se asignan al control:

cmbMaestros

Configuración del control:

Propiedad	Valor
DataSource	DataTable con maestros
DisplayMember	Nombre
ValueMember	IdMaestro

Consulta para cargar materias

SELECT IdMateria, Nombre FROM Materias

Estos datos se asignan al control:

cmbMaterias

Configuración:

Propiedad	Valor
DataSource	DataTable con materias
DisplayMember	Nombre
ValueMember	IdMateria

18.5 Evento btnGuardar_Click

Este evento se ejecuta cuando el usuario presiona el botón **Guardar**.

Su función es **actualizar la materia seleccionada asignándole el maestro seleccionado**.

Consulta utilizada:

UPDATE Materias SET IdMaestro = @idM WHERE IdMateria = @idMat

Parámetros utilizados

Parámetro Valor

@idM ID del maestro seleccionado

@idMat ID de la materia seleccionada

Estos valores se obtienen de los controles:

cmbMaestros.SelectedValue

cmbMaterias.SelectedValue

El uso de **parámetros SQL** evita problemas de seguridad como **inyección SQL**.

18.6 Flujo de Funcionamiento del Formulario

El funcionamiento del formulario puede describirse en los siguientes pasos:

1. El formulario se abre.
2. Se cargan los maestros desde la base de datos.
3. Se cargan las materias disponibles.

4. El usuario selecciona un maestro.
 5. El usuario selecciona una materia.
 6. El usuario presiona **Guardar**.
 7. El sistema actualiza la tabla Materias con el maestro asignado.
-

18.7 Manejo de Errores

El sistema implementa bloques try-catch para capturar errores durante la ejecución.

Ejemplo:

```
catch (Exception ex)
{
    MessageBox.Show("Error al asignar: " + ex.Message);
}
```

Esto permite mostrar un mensaje informativo en caso de:

- Error de conexión
 - Error en la consulta SQL
 - Datos inválidos
-

19. Archivo de Recursos FormAsignar.resx

El archivo FormAsignar.resx es un archivo de **recursos utilizado por Windows Forms** para almacenar información relacionada con la interfaz del formulario.

Los archivos .resx permiten guardar recursos como:

- Textos
 - Iconos
 - Imágenes
 - Configuración de controles
-

19.1 Propósito del Archivo

El archivo FormAsignar.resx permite separar los recursos del código fuente, facilitando:

- La **organización del proyecto**
- La **internacionalización del sistema**
- La **gestión de recursos gráficos**

19.2 Estructura del Archivo

El archivo está basado en **XML** y utiliza el esquema Microsoft ResX.

Ejemplo de encabezado:

```
<resheader name="resmimetype">  
  <value>text/microsoft-resx</value>  
</resheader>
```

Este encabezado indica que el archivo utiliza el formato estándar de recursos de Microsoft.

19.3 Componentes del Archivo

El archivo contiene varias secciones importantes:

Elemento	Descripción
resheader	Información sobre el formato del archivo
data	Recursos almacenados
assembly	Referencias a ensamblados .NET
metadata	Información adicional

19.4 Función dentro del Proyecto

En este proyecto, el archivo FormAsignar.resx se utiliza principalmente para:

- Mantener la configuración visual del formulario
- Guardar propiedades de los controles
- Gestionar recursos asociados a la interfaz

La mayor parte del contenido se genera **automáticamente por Visual Studio**, por lo que normalmente **no es necesario modificarlo manualmente**.

20. Relación con la Arquitectura del Sistema

El formulario FormAsignar forma parte del módulo de **gestión académica**, encargado de administrar la relación entre maestros y materias.

Dentro de la arquitectura del sistema:

- **Interfaz de usuario:** Windows Forms
- **Lógica de aplicación:** Código C#

- **Persistencia de datos:** Base de datos MySQL

Este formulario representa la capa de **interacción con el usuario**, permitiendo ejecutar operaciones sobre la base de datos mediante consultas SQL.

La **documentación técnica del archivo FormAsignar.Designer.cs**, redactada de forma clara para incluirla en la **documentación del proyecto**.

21. Archivo de Diseño del Formulario

FormAsignar.Designer.cs

El archivo FormAsignar.Designer.cs contiene la **definición visual del formulario** utilizado para asignar maestros a materias dentro del sistema **Gestor de Horarios de Maestros**.

Este archivo es generado automáticamente por el entorno de desarrollo **Microsoft Visual Studio** cuando se diseña la interfaz mediante el editor visual de **Windows Forms**.

Su función principal es **definir los controles de la interfaz gráfica**, sus propiedades y su posición dentro del formulario.

21.1 Propósito del Archivo

El archivo Designer.cs separa la **lógica del programa** de la **definición visual del formulario**.

Esto permite:

- Mantener el código organizado.
- Facilitar el mantenimiento del sistema.
- Permitir que el diseñador visual genere el código automáticamente.

El archivo funciona como **parte parcial de la clase FormAsignar**, utilizando la palabra clave `partial`.

21.2 Controles del Formulario

El formulario incluye los siguientes controles de interfaz:

Control	Tipo	Función
cmbMaestros	ComboBox	Permite seleccionar un maestro
cmbMaterias	ComboBox	Permite seleccionar una materia
btnGuardar	Button	Ejecuta la asignación
lbl1	Label	Texto descriptivo para maestros
lbl2	Label	Texto descriptivo para materias

21.3 Método InitializeComponent()

El método InitializeComponent() es el encargado de **crear y configurar todos los controles del formulario**.

Este método se ejecuta automáticamente cuando se crea una instancia del formulario.

Dentro de este método se realizan las siguientes acciones:

1. Creación de los controles.
2. Configuración de propiedades.
3. Posicionamiento dentro del formulario.
4. Asociación de eventos.

21.4 Label de Selección de Maestro

El control lbl1 muestra un texto que indica al usuario que debe seleccionar un maestro.

```
this.lbl1.Text = "Seleccionar Maestro:";
```

```
this.lbl1.Location = new System.Drawing.Point(20, 20);
```

```
this.lbl1.AutoSize = true;
```

Propiedades principales:

Propiedad	Valor
Texto	Seleccionar Maestro
Posición	(20, 20)
Ajuste automático Activado	

21.5 ComboBox de Maestros

El control cmbMaestros permite mostrar la lista de maestros disponibles.

```
this.cmbMaestros.Location = new System.Drawing.Point(20, 40);
```

```
this.cmbMaestros.Size = new System.Drawing.Size(240, 21);
```

```
this.cmbMaestros.DropDownStyle = ComboBoxStyle.DropDownList;
```

Propiedades principales:

Propiedad	Valor
-----------	-------

Posición	(20, 40)
----------	----------

Propiedad Valor

Tamaño 240 x 21

Tipo Lista desplegable

Estilo Solo selección (DropDownList)

Este control se llena con datos provenientes de la tabla **Maestros**.

21.6 Label de Selección de Materia

El control lbl2 muestra el texto que indica al usuario seleccionar una materia.

```
this.lbl2.Text = "Seleccionar Materia:";
```

```
this.lbl2.Location = new System.Drawing.Point(20, 80);
```

```
this.lbl2.AutoSize = true;
```

Propiedades:

Propiedad Valor

Texto Seleccionar Materia

Posición (20, 80)

21.7 ComboBox de Materias

El control cmbMaterias permite seleccionar una materia existente en el sistema.

```
this.cmbMaterias.Location = new System.Drawing.Point(20, 100);
```

```
this.cmbMaterias.Size = new System.Drawing.Size(240, 21);
```

```
this.cmbMaterias.DropDownStyle = ComboBoxStyle.DropDownList;
```

Propiedades principales:

Propiedad Valor

Posición (20, 100)

Tamaño 240 x 21

Tipo Lista desplegable

Estilo Solo selección

Este control se llena con datos provenientes de la tabla **Materias**.

21.8 Botón de Asignación

El botón btnGuardar ejecuta el proceso de asignación de maestro a materia.

```
this.btnGuardar.Text = "Realizar Asignación";
```

```
this.btnGuardar.Location = new System.Drawing.Point(70, 150);
```

```
this.btnGuardar.Size = new System.Drawing.Size(140, 30);
```

Propiedades:

Propiedad	Valor
Texto	Realizar Asignación
Posición	(70,150)
Tamaño	140 x 30

21.9 Evento del Botón

El botón se conecta con el método que ejecuta la lógica de asignación:

```
this.btnGuardar.Click += new System.EventHandler(this.btnGuardar_Click);
```

Este evento ejecuta el método btnGuardar_Click definido en el archivo FormAsignar.cs.

21.10 Configuración del Formulario

Las propiedades generales del formulario se establecen al final del método:

```
this.ClientSize = new System.Drawing.Size(284, 211);
```

```
this.Text = "Asignar Maestro";
```

```
this.StartPosition = FormStartPosition.CenterParent;
```

Configuración:

Propiedad	Valor
Tamaño	284 x 211
Título	Asignar Maestro
Posición inicial	Centrado

21.11 Controles Agregados al Formulario

Los controles se agregan al formulario mediante el método Controls.AddRange().

```
this.Controls.AddRange(new Control[]
```

```
{  
    cmbMaestros,  
    cmbMaterias,  
    btnGuardar,  
    lbl1,  
    lbl2  
});
```

Esto permite que todos los elementos se muestren dentro de la ventana.

21.12 Relación con Otros Archivos

El formulario FormAsignar está compuesto por tres archivos principales:

Archivo	Función
FormAsignar.cs	Lógica del formulario
FormAsignar.Designer.cs	Diseño de la interfaz
FormAsignar.resx	Recursos del formulario

Esta estructura es estándar en aplicaciones desarrolladas con **Windows Forms**.

documentados:

- FormAsignar.cs
 - FormAsignar.Designer.cs
 - FormAsignar.resx
-

22. Módulo de Registro de Datos

Formulario: FormAgregar

El formulario FormAgregar permite **registrar nuevos maestros y nuevas materias** dentro del sistema **Gestor de Horarios de Maestros**.

Este formulario está dividido en dos secciones mediante un TabControl:

- **Pestaña 1:** Registro de Maestro
 - **Pestaña 2:** Registro de Materia
-

22.1 Estructura del Formulario

Archivo: FormAgregar.Designer.cs

Este archivo define la **interfaz gráfica del formulario**.

Uso de TabControl

El formulario utiliza un control TabControl para separar las funcionalidades:

```
this.tabControl.Controls.Add(this.tabMaestro);
```

```
this.tabControl.Controls.Add(this.tabMateria);
```

Pestañas

Pestaña	Función
---------	---------

tabMaestro	Agregar maestro
------------	-----------------

tabMateria	Agregar materia
------------	-----------------

Controles de la pestaña Maestro

Control	Tipo	Función
txtNombreMaestro	TextBox	Nombre del maestro
lblNombreMaestro	Label	Texto descriptivo

Controles de la pestaña Materia

El formulario incluye múltiples campos para registrar una materia completa:

Campo	Control
ID Materia	txtIdMateria
Maestro	cmbMaestro
Nombre	txtNombreMateria
Días	txtDias
Hora	txtHora
Aula	txtAula
H/D Crédito	txtHDCredito
Sección	txtSeccion

Campo	Control
Días/Mes	txtDiasMes
Créditos	txtCredito
Total Créditos	txtTotalCredito
Inscritos	txtInscritos

Método auxiliar Colocar()

private void Colocar(Label l, string t, int x, int y, Control p)

Este método permite:

- Crear labels dinámicamente
- Posicionarlos en pantalla
- Reducir código repetitivo

Práctica de reutilización de código.

Botones del formulario

Botón	Función
btnGuardar	Guarda datos
btnCancelar	Cierra el formulario

22.2 Lógica del Formulario

Archivo: FormAgregar.cs

Este archivo contiene toda la **lógica del sistema**.

22.3 Carga de Maestros

Método:

CargarComboMaestros()

Función:

- Consulta la base de datos
- Obtiene lista de maestros
- Llena el ComboBox

Consulta SQL:

```
SELECT IdMaestro, Nombre FROM Maestros ORDER BY Nombre
```

22.4 Clase ComboItem

Se utiliza para manejar datos en el ComboBox:

```
public class ComboItem
```

Permite:

- Guardar ID y nombre
 - Mostrar nombre en pantalla
 - Usar ID internamente
-

22.5 Registro de Maestro

Método:

```
GuardarMaestro()
```

Validación

- El nombre no puede estar vacío

Inserción SQL

```
INSERT INTO Maestros (Nombre) VALUES (@nombre)
```

22.6 Validación de Horarios

Método clave:

```
ObtenerHorariosExistentes()
```

Función:

- Obtiene horarios actuales desde la base de datos
 - Convierte la hora a formato numérico
 - Construye una lista de objetos HorarioSimple
-

Procesamiento de hora

El sistema soporta formatos como:

- 08:00 - 10:00
- 08:00:00

Ejemplo:

```
horaDb.Split('-')[0].Trim()
```

22.7 Registro de Materia

Método:

GuardarMateria()

Paso 1: Validación de campos

Campos obligatorios:

- Maestro
 - Nombre de materia
 - Hora
-

Paso 2: Validación de choque de horarios

Se crea un objeto:

```
var nuevo = new HorarioSimple
```

Luego se valida con:

```
ValidadorHorarios.HayChoque()
```

Si hay conflicto:

- Se muestra mensaje
 - No se guarda
-

Paso 3: Inserción en base de datos

Consulta SQL:

```
INSERT INTO Materias (...)
```

```
VALUES (...)
```

Se insertan todos los campos de la materia.

Manejo de datos numéricos

Método:

ParseInt()

Función:

- Convierte texto a entero
- Si falla → guarda NULL

Esto evita errores de tipo en la base de datos.

22.8 Limpieza de Campos

Método:

LimpiarTabMateria()

Función:

- Limpia todos los campos del formulario
 - Mejora la experiencia del usuario
-

22.9 Evento Cancelar

BtnCancelar_Click()

Función:

- Cierra el formulario
-

22.10 Flujo de Funcionamiento

Para agregar maestro:

1. Usuario entra en pestaña "Nuevo Maestro"
 2. Escribe nombre
 3. Presiona Guardar
 4. Se inserta en la base de datos
-

Para agregar materia:

1. Usuario cambia a pestaña "Nueva Materia"
2. Se cargan los maestros
3. Llena los campos
4. Se valida el horario
5. Si no hay conflicto → se guarda
6. Se limpian los campos

22.11 Buenas Prácticas Aplicadas

Este formulario implementa varias buenas prácticas:

Separación de responsabilidades

- Diseño → Designer.cs
- Lógica → FormAgregar.cs

Validación de datos

- Campos obligatorios
- Validación de horarios

Seguridad

- Uso de parámetros SQL

Manejo de errores

- Uso de try-catch

Reutilización de código

- Método Colocar()
-

22.12 Observaciones Técnicas

- IdMateria se ingresa manualmente (riesgo de duplicados)
 - Validación de horarios es en memoria (no en BD)
 - Manejo flexible de formatos de hora
 - Sistema preparado para expansión futura
-

El formulario FormAgregar es uno de los componentes más importantes del sistema, ya que permite:

- Registrar nuevos datos
- Validar información
- Mantener la integridad de horarios

Además, combina correctamente:

- Interfaz gráfica
- Lógica de negocio
- Acceso a datos

Documentación del Formulario: FormAgregar

El formulario FormAgregar forma parte del sistema desarrollado en **Windows Forms** utilizando el entorno de desarrollo **Microsoft Visual Studio**. Su propósito es permitir el registro de nuevos maestros y materias dentro del sistema.

Este formulario está compuesto por tres componentes principales:

1. Diseño de la Interfaz

Archivo: FormAgregar.Designer.cs

Este archivo define la **estructura visual del formulario** y los controles gráficos utilizados.

Características principales:

- Uso de un **TabControl** con dos pestañas:
 - **Nuevo Maestro**
 - **Nueva Materia**

Controles principales:

◆ Pestaña "Nuevo Maestro"

- txtNombreMaestro: Campo para ingresar el nombre del maestro.
- lblNombreMaestro: Etiqueta descriptiva.

◆ Pestaña "Nueva Materia"

Incluye múltiples campos para registrar una materia:

Campo	Descripción
txtIdMateria	ID de la materia
cmbMaestro	Selección del maestro
txtNombreMateria	Nombre de la materia
txtDias	Días en que se imparte
txtHora	Horario
txtAula	Aula asignada
txtSeccion	Sección
txtHDCredito	Horas/Crédito

Campo	Descripción
txtDiasMes	Días por mes
txtCredito	Créditos
txtTotalCredito	Total de créditos
txtInscritos	Cantidad de estudiantes

Botones:

- btnGuardar: Guarda los datos.
- btnCancelar: Cierra el formulario.

Aspecto técnico:

- Se utiliza el método auxiliar Colocar() para posicionar etiquetas dinámicamente.
- La interfaz está organizada en forma de formulario estructurado para facilitar la entrada de datos.

2. Lógica del Formulario

Archivo: FormAgregar.cs

Este archivo contiene **la funcionalidad del sistema**, incluyendo la conexión a la base de datos y validaciones.

Funciones principales:

Conexión a la base de datos

Se utiliza una cadena de conexión definida en:

ConfigurationManager.ConnectionStrings["MiConexion"]

Y se conecta mediante:

MySqlConnection

Carga de datos dinámicos

Método:

CargarComboMaestros()

- Obtiene los maestros desde la base de datos.
 - Llena el ComboBox cmbMaestro.
 - Se ejecuta al cambiar de pestaña.
-

Registro de Maestro

Método:

GuardarMaestro()

Funciones:

- Valida que el nombre no esté vacío.
 - Inserta el maestro en la tabla Maestros.
 - Limpia el campo después de guardar.
-

Registro de Materia

Método:

GuardarMateria()

Este es el proceso más complejo del formulario.

Incluye:

1. Validaciones básicas:

- Maestro seleccionado
 - Nombre de materia
 - Hora definida
-

2. Validación de choque de horarios

Se utiliza:

- Clase HorarioSimple
- Método:

ObtenerHorariosExistentes()

Este método:

- Consulta la base de datos.
- Obtiene horarios existentes.
- Convierte la hora en formato numérico.
- Permite comparar horarios.

Luego se usa:

ValidadorHorarios.HayChoque()

Para evitar conflictos de horario entre materias.

3. Inserción en base de datos

Se ejecuta un INSERT INTO Materias con parámetros:

- ID de materia
- Maestro
- Nombre
- Días
- Hora
- Créditos
- Aula
- Sección
- Inscritos

◆ Métodos auxiliares

ParseInt()

Convierte valores a enteros o NULL para la base de datos.

LimpiarTabMateria()

Limpia todos los campos después de guardar.

◆ Clase auxiliar: Comboltem

Permite almacenar:

- ID del maestro
- Nombre visible en el ComboBox

Sobrescribe ToString() para mostrar correctamente el nombre.

3. Recursos del Formulario

Archivo: FormAgregar.resx

Este archivo contiene **los recursos del formulario en formato XML**.

Características:

- Define el esquema estándar de recursos de .NET.
- Incluye metadatos como:

- Versión del archivo
- Tipo de lector (ResXResourceReader)
- Tipo de escritor (ResXResourceWriter)

Función:

- Permite almacenar recursos como:
 - Textos
 - Imágenes
 - Iconos
 - Configuraciones del formulario

Importante:

En este caso específico:

- El archivo **no contiene recursos personalizados**, solo la estructura base.
- Es generado automáticamente por el sistema.

Conclusión de la Sección

El formulario FormAgregar implementa una arquitectura organizada basada en la separación de responsabilidades:

- **Interfaz gráfica (Designer)** → Define la estructura visual.
- **Lógica (CS)** → Controla el comportamiento y acceso a datos.
- **Recursos (RESX)** → Administra configuraciones y elementos externos.

Este enfoque permite:

- Mayor claridad en el desarrollo.
- Mantenimiento más sencillo.
- Escalabilidad del sistema.

23. Módulo de Configuración de Conexión

Archivo: ConfigConexion.Designer.cs

El archivo ConfigConexion.Designer.cs define la **interfaz gráfica del formulario de configuración de conexión a la base de datos** dentro del sistema **Gestor de Horarios de Maestros**.

Este formulario permite al usuario ingresar manualmente los parámetros necesarios para establecer la conexión con la base de datos **MySQL**.

23.1 Propósito del Formulario

El formulario ConfigConexion tiene como objetivo:

- Permitir al usuario configurar la conexión a la base de datos.
- Evitar la modificación directa del archivo app.config.
- Facilitar la configuración inicial del sistema.

Este módulo es clave dentro del sistema, ya que permite que la aplicación funcione en distintos entornos sin necesidad de cambiar el código fuente.

23.2 Controles del Formulario

El formulario contiene varios controles organizados de manera estructurada para capturar los datos de conexión.

Título del formulario

label1.Text = "Escribe la Ruta de la Conexión:";

- Muestra el encabezado principal.
 - Utiliza una fuente grande y en negrita para destacar.
-

Texto de ayuda

label2.Text = "Ejemplo: Server=TU_SERVIDOR; Database=TuBD; Integrated Security=True;";

- Proporciona un ejemplo de cadena de conexión.
 - Ayuda al usuario a introducir correctamente los datos.
-

◆ Campos de entrada

1. Servidor

txtServer

Propiedad Valor

Tipo TextBox

Función Dirección del servidor

2. Base de datos

txtDatabase

Propiedad Valor

Tipo TextBox

Función Nombre de la base de datos

3. Usuario

txtUser

Propiedad Valor

Tipo TextBox

Función Usuario de la base de datos

4. Contraseña

txtPassword

Propiedad Valor

Tipo TextBox

Función Contraseña

Seguridad Oculta caracteres (UseSystemPasswordChar = true)

◆ Botón Guardar

button1.Text = "Guardar";

- Ejecuta la acción de guardar la configuración.
- Está vinculado al evento:

button1_Click

23.3 Diseño de la Interfaz

El formulario presenta un diseño sencillo y funcional:

- Organización vertical de los campos.
- Etiquetas alineadas a la izquierda.
- Campos de entrada alineados a la derecha.
- Botón de acción ubicado en la parte inferior.

23.4 Propiedades del Formulario

ClientSize = new Size(610, 180);

Text = "ConfigConexion";

Propiedad Valor

Tamaño 610 x 180

Título ConfigConexion

23.5 Eventos Asociados

El formulario incluye eventos vinculados a los controles:

Control	Evento
button1	Click
txtServer	TextChanged
txtDatabase	TextChanged
txtUser	TextChanged
txtPassword	TextChanged

Estos eventos permiten capturar cambios y ejecutar lógica adicional (definida en el archivo .cs).

23.6 Importancia dentro del Sistema

Este formulario es fundamental porque:

- Permite configurar la conexión sin modificar código.
 - Facilita la instalación del sistema en diferentes equipos.
 - Mejora la usabilidad para usuarios no técnicos.
-

23.7 Buenas Prácticas Aplicadas

Separación de responsabilidades

Interfaz clara y amigable

Uso de campos específicos para cada parámetro

Protección de contraseña (oculta)

Uso de eventos para interacción dinámica

23.8 Posibles Mejoras

Aunque funcional, el formulario podría mejorarse con:

- Validación de campos obligatorios.
 - Botón “Probar conexión”.
 - Mensajes de confirmación.
 - Encriptación de la contraseña.
 - Uso de cadenas de conexión completas automáticas.
-

Conclusión del Módulo

Logrando una solución sencilla y efectiva para la configuración de la conexión a la base de datos.

Su diseño facilita la interacción del usuario y permite adaptar el sistema a distintos entornos sin necesidad de modificar el código fuente, lo que representa una buena práctica dentro del desarrollo de software y el **Gobierno de TI**.

La documentación del módulo **ConfigConexion** con su **lógica (archivo .cs)**, cerrando así el formulario completo (Diseño + Lógica).

24. Lógica del Formulario de Configuración

Archivo: ConfigConexion.cs

El archivo ConfigConexion.cs contiene la **lógica funcional** del formulario encargado de configurar la conexión a la base de datos en el sistema.

Este módulo trabaja junto con el archivo ConfigConexion.Designer.cs, que define la interfaz gráfica.

24.1 Propósito

Permitir al usuario:

- Crear una nueva cadena de conexión.
 - Guardarla directamente en el archivo de configuración (app.config).
 - Aplicar los cambios sin reiniciar la aplicación.
-

24.2 Construcción de la Cadena de Conexión

Dentro del evento principal:

```
string nuevaCadena    =  
$"Server={txtServer.Text};Database={txtDatabase.Text};Uid={txtUser.Text};Pwd={txtPassword.  
Text};";
```

Explicación:

Se construye dinámicamente una cadena de conexión para **MySQL**, utilizando los valores ingresados por el usuario en los campos del formulario.

Estructura:

```
Server=SERVIDOR;
```

```
Database=BASE_DE_DATOS;
```

```
Uid=USUARIO;
```

```
Pwd=CONTRASEÑA;
```

24.3 Guardado en el Archivo de Configuración

Método utilizado:

```
Configuration config    =  
ConfigurationManager.OpenExeConfiguration(ConfigurationUserLevel.None);
```

Función:

- Abre el archivo de configuración del ejecutable (.exe.config).
-

Actualización de la cadena de conexión:

```
config.ConnectionStrings.ConnectionStrings["MiConexion"].ConnectionString = nuevaCadena;
```

Función:

- Reemplaza la cadena de conexión existente llamada "**MiConexion**".
-

Guardado permanente:

```
config.Save(ConfigurationSaveMode.Modified);
```

Función:

- Guarda los cambios directamente en el archivo de configuración.
-

Recarga en memoria:

```
ConfigurationManager.RefreshSection("connectionStrings");
```

Función:

- Permite que la aplicación utilice inmediatamente la nueva configuración sin reiniciarse.
-

24.4 Evento Principal: Guardar

Método:

button1_Click(object sender, EventArgs e)

Flujo completo:

1. Captura los datos ingresados.
 2. Construye la cadena de conexión.
 3. Abre el archivo de configuración.
 4. Actualiza la cadena existente.
 5. Guarda los cambios.
 6. Recarga la configuración.
 7. Muestra mensaje de éxito.
 8. Cierra el formulario.
-

24.5 Manejo de Errores

El sistema implementa un bloque try-catch:

```
catch (Exception ex)
{
    MessageBox.Show("Error al guardar la configuración: " + ex.Message);
}
```

Función:

- Captura errores como:
 - Problemas de acceso al archivo
 - Configuración inexistente
 - Errores de escritura
 - Notifica al usuario mediante un mensaje.
-

24.6 Eventos Secundarios

El formulario incluye eventos adicionales:

Método	Función
label1_Click	Sin funcionalidad
label2_Click	Sin funcionalidad
label3_Click	Sin funcionalidad
txtServer_TextChanged	Sin funcionalidad
txtDatabase_TextChanged	Sin funcionalidad
txtUser_TextChanged	Sin funcionalidad
txtPassword_TextChanged	Sin funcionalidad

Nota:

Estos eventos están definidos pero no implementan lógica, lo cual es común en formularios generados automáticamente.

24.7 Importancia dentro del Sistema

Este módulo es clave porque:

- Permite **configuración dinámica** del sistema.
- Evita recompilar el programa para cambiar la conexión.
- Facilita la portabilidad del sistema.

24.8 Buenas Prácticas Aplicadas

Uso de ConfigurationManager para gestión de configuración

Separación entre interfaz y lógica

Manejo de excepciones

Aplicación de cambios en tiempo real

Uso de interpolación de cadenas (código limpio)M

Conclusión del Módulo

El archivo ConfigConexion.cs implementa una solución eficiente para la gestión dinámica de la conexión a la base de datos.

Su integración con el sistema permite mayor flexibilidad, facilidad de uso y adaptación a distintos entornos, lo cual es una práctica alineada con principios modernos de desarrollo y **Gobierno de TI**.

25 Archivo de Diseño del Formulario de Búsqueda

FormBuscar.Designer.cs

El archivo **FormBuscar.Designer.cs** define la interfaz gráfica del formulario utilizado para realizar búsquedas dentro del sistema Gestor de Horarios de Maestros.

Este formulario permite al usuario ingresar diferentes criterios para filtrar o localizar información relacionada con los horarios académicos.

25.1 Propósito del Formulario

El formulario **FormBuscar** tiene como objetivo:

- Permitir la búsqueda de horarios mediante filtros
 - Facilitar la localización de registros específicos
 - Mejorar la usabilidad del sistema
-

25.2 Controles del Formulario

El formulario incluye los siguientes controles:

Control	Tipo	Función
txtSeccion	TextBox	Ingresar sección
txtDia	TextBox	Ingresar día
txtCredito	TextBox	Ingresar crédito
txtHora	TextBox	Ingresar hora
lbl1	Label	Texto "Sección"
lbl2	Label	Texto "Día"
lbl3	Label	Texto "Crédito"
lbl4	Label	Texto "Hora"
btnAceptar	Button	Ejecuta la búsqueda
btnCancelar	Button	Cancela la operación

25.3 Campos de Entrada

El formulario permite ingresar múltiples criterios de búsqueda:

Sección (txtSeccion)

```
txtSeccion.Location = new Point(100, 17);
```

```
txtSeccion.Size = new Size(150, 23);
```

Permite filtrar resultados por sección académica.

Día (txtDia)

```
txtDia.Location = new Point(100, 47);
```

Permite buscar horarios según el día de clase.

Crédito (txtCredito)

```
txtCredito.Location = new Point(100, 77);
```

Permite filtrar por cantidad de créditos de la materia.

Hora (txtHora)

```
txtHora.Location = new Point(100, 107);
```

Permite buscar horarios en una hora específica.

25.4 Etiquetas Descriptivas

Las etiquetas guían al usuario en la entrada de datos:

Label Texto

lbl1 Sección

lbl2 Día

lbl3 Crédito

lbl4 Hora

Estas etiquetas están alineadas con sus respectivos campos para mejorar la claridad visual.

25.5 Botones de Acción

btnAceptar (Buscar)

```
btnAceptar.Text = "Buscar";
```

```
btnAceptar.Click += btnAceptar_Click;
```

Función:

- Recoge los datos ingresados
- Ejecuta la lógica de búsqueda
- Muestra los resultados

btnCancelar

btnCancelar.Text = "Cancelar";

btnCancelar.Click += btnCancelar_Click;

Función:

- Cierra el formulario
- Cancela la operación actual

25.6 Método InitializeComponent()

El método **InitializeComponent()** es responsable de:

- Crear los controles
- Configurar sus propiedades
- Definir su posición
- Asociar eventos

Este método se ejecuta automáticamente al abrir el formulario.

25.7 Configuración del Formulario

ClientSize = new Size(280, 200);

FormBorderStyle = FormBorderStyle.FixedDialog;

MaximizeBox = false;

MinimizeBox = false;

StartPosition = FormStartPosition.CenterParent;

Propiedades:

Propiedad	Valor
------------------	--------------

Tamaño	280 x 200
--------	-----------

Tipo de ventana	Diálogo fijo
-----------------	--------------

Propiedad	Valor
Redimensionable	No
Posición	Centrado

25.8 Organización Visual

El formulario presenta un diseño compacto y ordenado:

- Campos alineados verticalmente
- Etiquetas a la izquierda
- Entradas a la derecha
- Botones en la parte inferior

Esto facilita la interacción del usuario.

25.9 Relación con la Lógica del Sistema

El formulario trabaja junto con:

Archivo	Función
FormBuscar.cs	Lógica de búsqueda
FormBuscar.Designer.cs	Interfaz gráfica
FormBuscar.resx	Recursos

25.10 Posible Lógica de Búsqueda (Contexto)

Aunque este archivo no contiene lógica, los datos ingresados pueden utilizarse para construir consultas SQL dinámicas, por ejemplo:

```
SELECT * FROM HorariosView
```

```
WHERE Seccion = @seccion
```

```
AND Dia = @dia
```

```
AND Credito = @credito
```

```
AND Hora = @hora;
```

25.11 Buenas Prácticas Aplicadas

Separación de diseño y lógica
Interfaz clara y simple

Conclusión

Se permite realizar búsquedas específicas dentro del sistema, facilitando la consulta de información académica.

Su diseño simple y funcional lo convierte en una herramienta útil para mejorar la navegación y acceso a los datos.

▼ 26. Lógica del Formulario de Búsqueda

Archivo: FormBuscar.cs

El archivo **FormBuscar.cs** contiene la lógica del formulario encargado de capturar los criterios de búsqueda ingresados por el usuario dentro del sistema Gestor de Horarios de Maestros.

Este formulario no realiza directamente la consulta a la base de datos, sino que actúa como un **formulario de entrada de datos**, devolviendo los valores al formulario principal para que este ejecute la búsqueda.

26.1 Propósito del Archivo

El objetivo principal de este formulario es:

- Recopilar criterios de búsqueda
 - Enviar los datos al formulario principal
 - Controlar la interacción del usuario mediante botones
-

26.2 Propiedades de Comunicación

El formulario utiliza propiedades públicas para exponer los datos ingresados por el usuario:

```
public string Seccion => txtSeccion.Text;
```

```
public string Dia => txtDia.Text;
```

```
public string Credito => txtCredito.Text;
```

```
public string Hora => txtHora.Text;
```

Función

Estas propiedades permiten:

- Acceder a los valores desde otro formulario
- Evitar el uso de variables globales

- Mantener encapsulación de datos
-

26.3 Flujo de Uso del Formulario

El formulario funciona como un cuadro de diálogo (Dialog Form):

1. El formulario principal abre **FormBuscar**
 2. El usuario ingresa los criterios
 3. El usuario presiona "Buscar" o "Cancelar"
 4. El formulario devuelve un resultado (OK o Cancel)
 5. El formulario principal recibe los datos
-

26.4 Constructor del Formulario

```
public FormBuscar()  
{  
    InitializeComponent();  
    this.StartPosition = FormStartPosition.CenterParent;  
    this.Text = "Criterios de Búsqueda";  
}
```

Funciones del constructor:

- Inicializa los componentes gráficos
 - Define la posición centrada del formulario
 - Establece el título de la ventana
-

26.5 Evento btnAceptar_Click

```
private void btnAceptar_Click(object sender, EventArgs e)  
{  
    this.DialogResult = DialogResult.OK;  
    this.Close();  
}
```

Función:

- Indica que el usuario confirmó la búsqueda
- Cierra el formulario

- Permite al formulario principal continuar con la consulta

26.6 Evento btnCancelar_Click

```
private void btnCancelar_Click(object sender, EventArgs e)
{
    this.DialogResult = DialogResult.Cancel;
    this.Close();
}
```

Función:

- Indica que el usuario canceló la operación
- Cierra el formulario sin enviar datos

26.7 Uso desde el Formulario Principal

El formulario **FormBuscar** se utiliza normalmente de la siguiente manera:

```
using (var f = new FormBuscar())
{
    if (f.ShowDialog() == DialogResult.OK)
    {
        string seccion = f.Seccion;
        string dia = f.Dia;
        string credito = f.Credito;
        string hora = f.Hora;

        // Aquí se ejecuta la búsqueda en la base de datos
    }
}
```

26.8 Tipo de Formulario

Este formulario funciona como:

Cuadro de diálogo modal
Captura de parámetros
Interfaz de filtrado

26.9 Ventajas del Diseño

Separación de responsabilidades
Reutilización del formulario
Código limpio y organizado
Fácil integración con otros módulos

26.10 Buenas Prácticas Aplicadas

- Uso de propiedades en lugar de variables públicas
 - Uso de DialogResult para control de flujo
 - Separación entre interfaz y lógica de búsqueda
 - Diseño desacoplado del acceso a datos
-

Conclusión

Cumpliendo correctamente su función como componente de captura de criterios de búsqueda, permitiendo una interacción simple y efectiva con el usuario.

Su diseño desacoplado facilita su integración con el formulario principal, donde se ejecuta la lógica de consulta.

27. Archivo de Recursos del Formulario

FormBuscar.resx

El archivo **FormBuscar.resx** es un archivo de recursos asociado al formulario **FormBuscar**, utilizado en aplicaciones desarrolladas con Windows Forms en .NET.

Este archivo permite almacenar información adicional del formulario en formato XML, separando los recursos del código fuente.

27.1 Propósito del Archivo

Hay que recordar que los archivos .resx son archivos de recursos utilizados en aplicaciones desarrolladas con Windows Forms en .NET. Estos archivos permiten almacenar información adicional de los formularios en formato XML, separando los recursos visuales del código fuente y algunos archivos pueden compartir las mismas funciones que otros.

El archivo **.resx** tiene como objetivo:

- Almacenar recursos del formulario
- Separar datos del código fuente

- Facilitar la internacionalización del sistema
 - Mantener la configuración del formulario organizada
-

27.2 Tipo de Archivo

El archivo utiliza el formato estándar:

Microsoft ResX (Resource XML)

Este formato es:

Basado en XML

Legible por humanos

Procesado automáticamente por .NET

27.3 Estructura del Archivo

El archivo está compuesto por varias secciones principales:

Encabezado (resheader)

Define información general del archivo:

```
<resheader name="resmimetype">  
  <value>text/microsoft-resx</value>  
</resheader>
```

Otros encabezados incluyen:

- version → versión del formato
 - reader → clase utilizada para leer recursos
 - writer → clase utilizada para escribir recursos
-

Esquema XML (xsd:schema)

Define la estructura del archivo:

- Tipos de datos permitidos
 - Organización de los elementos
 - Validación del contenido
-

Elementos data

Permiten almacenar recursos como:

- Textos

- Imágenes
- Iconos
- Datos serializados

Ejemplo:

```
<data name="TextoEjemplo">
  <value>Contenido</value>
</data>
```

27.4 Contenido del Archivo en el Proyecto

En este caso específico, el archivo **FormBuscar.resx**:

- No contiene recursos personalizados
 - Solo incluye la estructura base generada por Visual Studio
 - Funciona como archivo de soporte para el formulario
-

27.5 Función dentro del Sistema

Aunque no contiene datos visibles, el archivo cumple funciones importantes:

Mantiene compatibilidad con el diseñador visual
 Permite agregar recursos en el futuro
 Soporta propiedades del formulario
 Facilita mantenimiento del proyecto

27.6 Relación con Otros Archivos

El formulario **FormBuscar** está compuesto por:

Archivo	Función
FormBuscar.cs	Lógica del formulario
FormBuscar.Designer.cs	Interfaz gráfica
FormBuscar.resx	Recursos del formulario

27.7 Ventajas del Uso de Archivos .resx

Separación de responsabilidades
 Mejora la organización del proyecto
 Permite localización (multi-idioma)
 Facilita reutilización de recursos

27.8 Generación Automática

El archivo es:

- Generado automáticamente por Visual Studio
- Actualizado al modificar el formulario
- No requiere edición manual en la mayoría de los casos

Conclusión del Módulo FormBuscar

El formulario **FormBuscar** mantiene el modelo estándar de Windows Forms:

- **FormBuscar.cs** → Lógica
- **FormBuscar.Designer.cs** → Interfaz
- **FormBuscar.resx** → Recursos

Esta organización permite:

- Código limpio
- Fácil mantenimiento
- Escalabilidad del sistema

▼ 28. Archivo de Diseño del Formulario de Impresión

FormImprimir.Designer.cs

El archivo **FormImprimir.Designer.cs** define la interfaz gráfica del formulario encargado de visualizar e imprimir los horarios de los maestros dentro del sistema.

Este formulario permite:

- Seleccionar un maestro
- Visualizar sus horarios
- Generar un reporte imprimible

28.1 Propósito del Formulario

El formulario **FormImprimir** tiene como objetivo:

- Mostrar los horarios de un maestro específico
- Permitir generar reportes
- Facilitar la impresión de información académica

28.2 Controles del Formulario

El formulario incluye los siguientes elementos:

Control	Tipo	Función
comboBox1	ComboBox	Selección de maestro
button1	Button	Imprimir reporte
dataGridView1	DataGridView	Mostrar horarios
label1	Label	Texto "Maestro"

28.3 ComboBox de Selección

comboBox1

```
comboBox1.Location = new Point(73, 14);
```

```
comboBox1.Size = new Size(171, 23);
```

```
comboBox1.SelectedIndexChanged += comboBox1_SelectedIndexChanged;
```

Función:

- Permite seleccionar un maestro
 - Al cambiar la selección, se actualiza la tabla de horarios
-

28.4 Botón de Impresión

button1

```
button1.Text = "Imprimir Reporte";
```

```
button1.Click += button1_Click;
```

Función:

- Ejecuta el proceso de impresión
 - Genera un reporte del maestro seleccionado
-

28.5 DataGridView de Visualización

dataGridView1

```
dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
```

```
dataGridView1.Anchor = AnchorStyles.Top | AnchorStyles.Bottom | AnchorStyles.Left |  
AnchorStyles.Right;
```

Función:

- Muestra los horarios del maestro
 - Ajusta automáticamente el tamaño de las columnas
 - Se adapta al tamaño del formulario
-

28.6 Etiqueta Descriptiva

label1

label1.Text = "Maestro:";

Función:

- Indica el propósito del ComboBox
 - Mejora la comprensión de la interfaz
-

28.7 Método InitializeComponent()

Este método se encarga de:

- Crear los controles
- Configurar propiedades
- Posicionar elementos
- Asociar eventos

Se ejecuta automáticamente al iniciar el formulario.

28.8 Evento de Carga del Formulario

Load += FormImprimir_Load;

Función:

- Se ejecuta al abrir el formulario
 - Permite cargar los datos iniciales
 - Generalmente se usa para llenar el ComboBox con maestros
-

28.9 Configuración del Formulario

ClientSize = new Size(553, 270);

StartPosition = FormStartPosition.CenterParent;

Text = "Imprimir";

Propiedades:

Propiedad Valor

Tamaño 553 x 270

Título Imprimir

Posición Centrado

28.10 Diseño Responsivo

El uso de **Anchor** en el DataGridView permite:

Ajuste automático al redimensionar

Mejor experiencia de usuario

Visualización adaptable

28.11 Flujo de Funcionamiento

El formulario sigue el siguiente flujo:

1. Se abre el formulario
 2. Se cargan los maestros en el ComboBox
 3. El usuario selecciona un maestro
 4. Se muestran sus horarios en la tabla
 5. El usuario presiona "Imprimir Reporte"
 6. Se genera el reporte
-

28.12 Relación con Otros Archivos

El formulario está compuesto por:

Archivo	Función
FormImprimir.cs	Lógica del formulario
FormImprimir.Designer.cs	Interfaz gráfica
FormImprimir.resx	Recursos

28.13 Buenas Prácticas Aplicadas

Separación de diseño y lógica

Uso de eventos para interacción

Uso de DataGridView para visualización
Interfaz clara y funcional

28.14 Importancia del Módulo

Este formulario añade una funcionalidad clave al sistema:

Generación de reportes
Visualización estructurada de datos
Soporte para impresión

Esto lo convierte en un módulo de **nivel profesional**, ya que los reportes son esenciales en sistemas reales.

Funcionalidad de impresión

- Exportar a PDF
 - Vista previa antes de imprimir
 - Uso de PrintDocument
-

Interfaz

- Agregar título del reporte
 - Mostrar fecha de impresión
 - Mejorar formato visual
-

Conclusión

Se permite visualizar e imprimir los horarios de los maestros, proporcionando una herramienta esencial para la gestión académica.

Su integración con la base de datos y su capacidad de generar reportes lo convierten en un componente clave.

29. Lógica del Formulario de Impresión y Reportes

Archivo: FormImprimir.cs

El archivo **FormImprimir.cs** contiene la lógica del módulo encargado de visualizar, filtrar y exportar los horarios de los maestros en formato PDF.

Este formulario representa uno de los componentes más avanzados del sistema, ya que integra:

- Acceso a datos
- Filtrado dinámico

- Compatibilidad con múltiples bases de datos
 - Generación de reportes
-

29.1 Propósito del Módulo

El objetivo principal de este formulario es:

- Mostrar horarios de maestros
 - Filtrar información por docente
 - Generar reportes en PDF
 - Permitir impresión de datos
-

29.2 Conexión Híbrida a Base de Datos

El sistema implementa un mecanismo de conexión adaptable que permite trabajar con:

- **SQLite (modo local)**
- **MySQL (modo servidor)**

Propiedad de configuración

```
private bool ModoLocal => Properties.Settings.Default.ModoConexion == "Local";
```

Método CrearConexion()

```
private IDbConnection CrearConexion()
```

Función:

- Determina el tipo de conexión
 - Crea una instancia de conexión según el modo
 - Permite flexibilidad en el despliegue del sistema
-

29.3 Ventajas de la Conexión Híbrida

Permite uso sin internet (modo local)

Facilita migración a servidor

Mejora portabilidad del sistema

Compatible con múltiples entornos

29.4 Creación de Parámetros SQL

```
private void CrearParametro(IDbCommand cmd, string nombre, object valor)
```

Función:

- Agregar parámetros a consultas SQL
 - Evitar inyección SQL
 - Manejar valores nulos correctamente
-

29.5 Carga de Datos para el Reporte

Método: `CargarDatosParaReporte()`

```
private void CargarDatosParaReporte(string filtroMaestro = "")
```

Función:

- Consulta la vista **HorariosView**
- Aplica filtro por maestro
- Llena un DataTable
- Muestra los datos en el DataGridView

Consulta base:

```
SELECT MaestroNombre, MateriaNombre, Seccion, DiasImparte, Hora, Aula  
FROM HorariosView
```

Filtro dinámico:

```
AND MaestroNombre LIKE @nombre
```

29.6 Llenado de Filtros

Método: `LlenarFiltros()`

Función:

- Obtiene lista de maestros únicos
- Llena el ComboBox
- Agrega opción "Todo"

Esto permite filtrar los datos de forma dinámica.

29.7 Exportación a PDF

Evento: `button1_Click`

Este método genera un archivo PDF con los datos mostrados en el DataGridView.

Proceso de generación

1. Se abre un cuadro de diálogo (SaveFileDialog)
 2. Se crea el documento PDF
 3. Se agrega un título
 4. Se inserta la fecha
 5. Se construye una tabla
 6. Se recorren los datos
 7. Se guarda el archivo
-

Librería utilizada

iTextSharp.text

iTextSharp.text.pdf

Elementos del PDF

Título del reporte

Fecha de generación

Tabla con datos

Encabezados de columnas

Ejemplo de encabezado

"Reporte del Gestor de Horarios Universitario"

29.8 Manejo de Errores

El sistema utiliza bloques try-catch para:

- Errores de conexión
- Fallos en consultas
- Problemas al generar PDF

Ejemplo:

```
catch (Exception ex)
```

```
{
```

```
    MessageBox.Show("Error: " + ex.Message);
```

```
}
```

29.9 Eventos del Formulario

FormImprimir_Load

LlenarFiltros();

CargarDatosParaReporte("");

Función:

- Cargar datos iniciales
 - Inicializar filtros
-

comboBox1_SelectedIndexChanged

Función:

- Detectar cambio de maestro
 - Aplicar filtro automáticamente
 - Actualizar tabla
-

29.10 Flujo de Funcionamiento

1. Se abre el formulario
 2. Se cargan los maestros
 3. Se muestran todos los horarios
 4. El usuario selecciona un maestro
 5. Se filtran los datos
 6. El usuario presiona "Imprimir"
 7. Se genera el PDF
-

29.11 Buenas Prácticas Aplicadas

Uso de interfaces (IDbConnection)

Separación de lógica y diseño

Uso de parámetros SQL

Manejo de errores

Reutilización de código

Arquitectura flexible

29.12 Nivel Técnico del Módulo

Este módulo implementa características de nivel intermedio-avanzado:

- Abstracción de base de datos
 - Generación de reportes
 - Manejo de datos dinámicos
 - Integración de librerías externas
-

Conclusión del Módulo

Representa uno de los componentes más completos del sistema, integrando:

- Consulta de datos
- Filtrado dinámico
- Generación de reportes

Su implementación demuestra un nivel sólido en el desarrollo de aplicaciones empresariales.

▼ 30. Archivo de Recursos del Formulario de Impresión

FormImprimir.resx

El archivo **FormImprimir.resx** es el archivo de recursos asociado al formulario **FormImprimir**, utilizado para almacenar información adicional del formulario en formato XML.

Este archivo forma parte de la estructura estándar de aplicaciones Windows Forms en .NET.

30.1 Propósito del Archivo

El siguiente archivo **.resx** tiene funciones principales que se han utilizado en este archivo:

- Almacenar recursos del formulario
 - Separar datos del código fuente
 - Facilitar la gestión de propiedades visuales
 - Permitir futuras extensiones del sistema
-

30.2 Formato del Archivo

El archivo utiliza el formato:

Microsoft Resource XML (ResX)

Siendo un contenedor de recursos que se utiliza para separar todo lo que "se ve" de la lógica de programación (el código C#).

Características:

Basado en XML que almacena objetos y datos estáticos
Legible y estructurado
Compatible con .NET Framework

30.3 Estructura del Archivo

El archivo contiene varias secciones clave:

Encabezados (resheader)

Definen la información general del archivo:

```
<resheader name="resmimetype">  
  <value>text/microsoft-resx</value>  
</resheader>
```

Otros encabezados incluyen:

- version → versión del formato
 - reader → lector de recursos
 - writer → escritor de recursos
-

Esquema XML (xsd:schema)

Define:

- Tipos de datos permitidos
 - Organización del archivo
 - Validación de contenido
-

Elementos data

Permiten almacenar:

- Textos
- Imágenes
- Iconos
- Objetos serializados

Ejemplo:

```
<data name="Ejemplo">  
  <value>Texto</value>
```

</data>

30.4 Contenido del Archivo en el Proyecto

En este caso, el archivo:

- No contiene recursos personalizados
- Solo incluye la estructura base
- Es generado automáticamente por Visual Studio

30.5 Función dentro del Sistema

Aunque no tenga contenido visible, el archivo cumple funciones importantes:

Soporte al diseñador visual
Base para agregar recursos futuros
Compatibilidad con el framework
Organización del proyecto

30.6 Relación con Otros Archivos

El formulario **FormImprimir** está compuesto por:

Archivo	Función
FormImprimir.cs	Lógica del sistema
FormImprimir.Designer.cs	Interfaz gráfica
FormImprimir.resx	Recursos

30.7 Ventajas del Uso de .resx

Separación de responsabilidades
Facilita mantenimiento
Permite internacionalización
Soporte para recursos complejos

Conclusión del Módulo FormImprimir

El formulario **FormImprimir** está correctamente estructurado siguiendo el modelo estándar:

- **FormImprimir.cs** → Lógica avanzada
- **FormImprimir.Designer.cs** → Interfaz

- **FormImprimir.resx** → Recursos

Este módulo destaca por:

Generación de reportes en PDF

Conexión híbrida a base de datos

Visualización dinámica de información

31. Archivo de Recursos del Formulario de Impresión

FormImprimir.resx

El archivo **FormImprimir.resx** es el archivo de recursos asociado al formulario **FormImprimir**, utilizado para almacenar información adicional del formulario en formato XML.

Este archivo forma parte de la estructura estándar de aplicaciones Windows Forms en .NET.

31.1 Propósito del Archivo

El archivo **.resx** para las funciones de imprimir se utilizaron las siguientes indicadas:

- Almacenar recursos del formulario
 - Separar datos del código fuente
 - Facilitar la gestión de propiedades visuales
 - Permitir futuras extensiones del sistema
-

31.2 Formato del Archivo

El archivo mantiene el formato:

Microsoft Resource XML (ResX)

Características:

Basado en XML

Legible y estructurado

Compatible con .NET Framework

31.3 Estructura del Archivo

El archivo contiene varias secciones clave:

Encabezados (resheader)

Definen la información general del archivo:

```
<resheader name="resmimetype">
```

```
<value>text/microsoft-resx</value>
```

```
</resheader>
```

Otros encabezados incluyen:

- version → versión del formato
 - reader → lector de recursos
 - writer → escritor de recursos
-

Esquema XML (xsd:schema)

Define:

- Tipos de datos permitidos
 - Organización del archivo
 - Validación de contenido
-

Elementos data

Permiten almacenar:

- Textos
- Imágenes
- Iconos
- Objetos serializados

Ejemplo:

```
<data name="Ejemplo">
```

```
<value>Texto</value>
```

```
</data>
```

31.4 Contenido del Archivo en el Proyecto

En este caso, el archivo:

- No contiene recursos personalizados
 - Solo incluye la estructura base
 - Es generado automáticamente por Visual Studio
-

31.5 Función dentro del Sistema

Aunque no tenga contenido visible, el archivo cumple funciones importantes:

Soporte al diseñador visual
Base para agregar recursos futuros
Compatibilidad con el framework
Organización del proyecto

31.6 Relación con Otros Archivos

El formulario **FormImprimir** está compuesto por:

Archivo	Función
FormImprimir.cs	Lógica del sistema
FormImprimir.Designer.cs	Interfaz gráfica
FormImprimir.resx	Recursos

31.7 Ventajas del Uso de .resx

Separación de responsabilidades
Facilita mantenimiento
Permite internacionalización
Soporte para recursos complejos

31.8 Generación y Mantenimiento

El archivo:

- Es creado automáticamente por Visual Studio
 - Se actualiza al modificar el formulario
 - No requiere edición manual
-

31.9 Recomendaciones de Uso

Para mejorar el sistema, se recomienda:

- Almacenar textos reutilizables
 - Incluir imágenes o logos institucionales
 - Preparar el sistema para múltiples idiomas
 - Centralizar recursos visuales
-

Conclusión del Módulo FormImprimir

El formulario **FormImprimir** está correctamente estructurado siguiendo el modelo estándar:

- **FormImprimir.cs** → Lógica avanzada
- **FormImprimir.Designer.cs** → Interfaz
- **FormImprimir.resx** → Recursos

Este módulo destaca por:

Generación de reportes en PDF
Conexión híbrida a base de datos
Visualización dinámica de información

32. Lógica del Formulario de Modificación

Archivo: FormModificar.cs

El archivo **FormModificar.cs** contiene la lógica encargada de gestionar la modificación de registros dentro del sistema, incluyendo la actualización de maestros y materias.

Este módulo implementa la operación **UPDATE del CRUD**, permitiendo editar información existente en la base de datos de manera controlada.

32.1 Propósito del Módulo

El formulario **FormModificar** tiene como objetivo:

- Modificar datos de maestros
 - Editar información completa de materias
 - Validar conflictos de horarios
 - Mantener la integridad de los datos
-

32.2 Conexión Híbrida a Base de Datos

El sistema permite trabajar con dos motores:

- SQLite (modo local)
- MySQL (modo servidor)

Propiedad de configuración

```
private bool ModoLocal => Properties.Settings.Default.ModoConexion == "Local";
```

Método CrearConexion()

Función:

- Determina el tipo de conexión

- Crea la conexión correspondiente
 - Permite flexibilidad en el sistema
-

32.3 Creación de Parámetros SQL

private void CrearParametro(IDbCommand cmd, string nombre, object valor)

Función:

- Agregar parámetros a consultas SQL
 - Prevenir inyección SQL
 - Manejar valores nulos
-

32.4 Carga Inicial de Datos

Método: CargarCombosIniciales()

CargarComboMaestros(cmbMaestros);

CargarComboMaestros(cmbMaestroAsoc);

CargarComboMaterias();

Función:

- Llenar ComboBox de maestros
 - Llenar ComboBox de materias
 - Preparar el formulario para edición
-

32.5 Carga de Combos

CargarComboMaestros()

- Consulta tabla **Maestros**
- Llena el ComboBox con objetos personalizados (CombolItem)

CargarComboMaterias()

- Consulta tabla **Materias**
 - Llena el ComboBox de materias disponibles
-

32.6 Uso de CombolItem

Se utiliza un objeto personalizado:

- ID (clave primaria)

- Nombre (texto visible)

Esto permite:

Manejar datos internos y visuales

Facilitar consultas posteriores

32.7 Carga de Datos de Materia

Evento: CmbMaterias_SelectedIndexChanged

Función:

- Detectar selección de materia
- Consultar base de datos
- Cargar datos en los campos del formulario

Campos cargados:

- ID
 - Nombre
 - Días
 - Hora
 - Créditos
 - Aula
 - Sección
 - Maestro asociado
-

32.8 Validación de Horarios

Método: ObtenerHorariosExistentes()

Función:

- Obtener horarios actuales (excepto el que se edita)
 - Convertir horas a formato numérico
 - Preparar datos para validación
-

Uso del Validador

ValidadorHorarios.HayChoque(...)

Función:

- Detectar conflictos de horario
 - Evitar duplicidad o solapamiento
 - Garantizar consistencia del sistema
-

32.9 Guardado de Datos

Evento principal

BtnGuardar_Click

Función:

- Determinar pestaña activa
 - Ejecutar método correspondiente
-

32.10 Actualización de Maestro

Método: GuardarMaestro()

UPDATE Maestros SET Nombre=@n WHERE IdMaestro=@id

Función:

- Actualizar nombre del maestro
 - Refrescar datos en interfaz
-

32.11 Actualización de Materia

Método: GuardarMateria()

Este método es el más complejo del módulo.

Proceso:

1. Validar selección de materia y maestro
 2. Obtener horarios existentes
 3. Convertir hora ingresada
 4. Validar conflicto de horario
 5. Ejecutar actualización
-

Consulta SQL

UPDATE Materias SET

IdMateria=@newId,
IdMaestro=@idM,
Nombre=@nom,
DiasImparte=@d,
Hora=@h,
HD_Credito=@hd,
DiasMes=@dm,
TotalCredito=@tot,
Inscritos=@ins,
Aula=@a,
Seccion=@s,
Credito=@c
WHERE IdMateria=@oldId

32.12 Conversión de Datos

Método: ParseInt()

private object ParseInt(string v)

Función:

- Convertir texto a número
 - Manejar valores inválidos como NULL
-

32.13 Eventos del Formulario

◆ CmbMaestros_SelectedIndexChanged

- Carga el nombre actual del maestro

◆ BtnCancelar_Click

- Cierra el formulario
-

32.14 Manejo de Errores

El sistema utiliza bloques try-catch para:

Errores de conexión

Fallos en consultas

Problemas de validación

32.15 Flujo de Funcionamiento

1. Se abre el formulario
2. Se cargan maestros y materias
3. El usuario selecciona una opción
4. Se cargan los datos
5. Se modifican los campos
6. Se valida el horario
7. Se guarda la información

32.16 Buenas Prácticas Aplicadas

Uso de parámetros SQL

Validación de datos

Separación de responsabilidades

Reutilización de métodos

Conexión flexible

Manejo de errores

32.17 Nivel Técnico del Módulo

Este módulo es de **nivel avanzado**, ya que incluye:

- Validación lógica compleja
 - Manejo de múltiples entidades
 - Integración de base de datos
 - Prevención de errores de negocio
-

32.18 Importancia dentro del Sistema

Este módulo representa:

La operación **UPDATE del CRUD**

Completa el ciclo del sistema junto con:

Operación Formulario

CREATE FormAgregar

READ FormBuscar

Operación Formulario

UPDATE FormModificar

REPORT FormImprimir

Conclusión del Módulo

El formulario **FormModificar** es un componente clave del sistema, encargado de mantener actualizada la información académica mediante procesos seguros y validados.

Su implementación demuestra un alto nivel de dominio en desarrollo de aplicaciones con base de datos.

33. Archivo de Recursos del Formulario de Modificación

FormModificar.resx

El archivo **FormModificar.resx** es un archivo de recursos asociado al formulario **FormModificar**, encargado de almacenar información adicional utilizada por la interfaz gráfica.

33.1 Propósito del Archivo

El archivo .resx de modificar se tuvo que implementar las siguientes funciones:

- Almacenar recursos del formulario
- Facilitar la internacionalización
- Separar contenido visual del código

33.2 Tipo de Contenido

En este caso específico, el archivo contiene únicamente:

- Configuración estándar de recursos
- Metadatos del sistema

No incluye:

Imágenes

Iconos

Cadenas personalizadas

33.3 Estructura del Archivo

El archivo sigue el formato XML estándar de .NET:

Encabezados principales

```
<resheader name="resmimetype">  
  <value>text/microsoft-resx</value>  
</resheader>
```

Define el tipo de archivo.

```
<resheader name="version">  
  <value>2.0</value>  
</resheader>
```

Indica la versión del esquema.

```
<resheader name="reader">
```

Define el lector de recursos.

```
<resheader name="writer">
```

Define el escritor de recursos.

33.4 Esquema XML (XSD)

El archivo incluye un esquema XML que define:

- Tipos de datos permitidos
- Estructura del documento
- Validación de contenido

Esto garantiza que los recursos sean interpretados correctamente por el sistema.

33.5 Funcionamiento dentro del Sistema

El archivo .resx:

- Es utilizado automáticamente por el formulario
- Se carga en tiempo de ejecución
- No requiere manipulación manual

33.6 Importancia en la Aplicación

Aunque este archivo no contiene recursos personalizados, sigue siendo importante porque:

Forma parte de la estructura del formulario

Permite agregar recursos en el futuro

Mantiene compatibilidad con el diseñador de Visual Studio

33.7 Relación con Otros Archivos

Archivo	Función
FormModificar.cs	Lógica del formulario
FormModificar.Designer.cs	Diseño de interfaz
FormModificar.resx	Recursos

33.8 Buenas Prácticas

El archivo logra cumplir las siguientes funciones

Mantener el archivo sin modificaciones manuales innecesarias

Utilizarlo para recursos reutilizables

Evitar duplicar información en código

33.9 Conclusión del Archivo

El archivo **FormModificar.resx** cumple un rol estructural dentro del formulario, asegurando la correcta gestión de recursos y compatibilidad con el entorno de desarrollo.

34. Archivo de Configuración del Proyecto

Archivo: Gestor de Horarios de Maestros.csproj

El archivo .csproj es el archivo principal de configuración del proyecto en aplicaciones desarrolladas con .NET. Define las propiedades del proyecto, las dependencias externas, la plataforma de ejecución y otros aspectos necesarios para la compilación y ejecución del sistema.

34.1 Propósito del Archivo

El archivo .csproj cumple las siguientes funciones:

- Define el tipo de aplicación (escritorio, web, etc.)

- Especifica la versión del framework .NET utilizada
- Configura el entorno de ejecución
- Gestiona las dependencias (librerías externas)
- Controla la compilación del proyecto

Este archivo es esencial para que el proyecto pueda ser compilado correctamente en Visual Studio.

34.2 Configuración General del Proyecto

Dentro del bloque <PropertyGroup> se definen las propiedades principales:

```
<PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFramework>net10.0-windows</TargetFramework>
  <RootNamespace>Gestor_de_Horarios_de_Maestros</RootNamespace>
  <Nullable>enable</Nullable>
  <UseWindowsForms>true</UseWindowsForms>
  <ImplicitUsings>enable</ImplicitUsings>
  <PlatformTarget>x86</PlatformTarget>
</PropertyGroup>
```

Descripción de cada propiedad:

Propiedad	Descripción
OutputType	Define que es una aplicación de escritorio (Windows Forms)
TargetFramework	Indica que el proyecto utiliza .NET 10 para Windows
RootNamespace	Espacio de nombres principal del proyecto
Nullable	Habilita el control de valores nulos para mayor seguridad
UseWindowsForms	Activa el uso de Windows Forms
ImplicitUsings	Permite usar librerías comunes sin declararlas manualmente
PlatformTarget	Define la arquitectura de ejecución (32 bits - x86)

34.3 Dependencias del Proyecto (PackageReference)

El proyecto utiliza varias bibliotecas externas para ampliar sus funcionalidades:

```

<ItemGroup>

  <PackageReference Include="iTextSharp" Version="5.5.13.5" />

  <PackageReference Include="MySQL.Data" Version="9.6.0" />

  <PackageReference Include="MySQLConnector" Version="2.5.0" />

  <PackageReference Include="System.Configuration.ConfigurationManager" Version="10.0.3" />

  <PackageReference Include="System.Data.SqlClient" Version="4.9.1" />

  <PackageReference Include="System.Data.SQLite.Core" Version="1.0.119" />

</ItemGroup>

```

Descripción de las librerías:

Librería	Función
iTextSharp	Permite generar archivos PDF (reportes del sistema)
MySQL.Data	Conector oficial para bases de datos MySQL
MySQLConnector	Alternativa moderna para conexión a MySQL
System.Configuration.ConfigurationManager	Permite leer configuraciones desde app.config
System.Data.SqlClient	Conexión a bases de datos SQL Server
System.Data.SQLite.Core	Soporte para base de datos SQLite

Observación técnica:

El proyecto incluye soporte para múltiples motores de base de datos (MySQL, SQL Server y SQLite), lo que lo hace flexible y adaptable.

34.4 Archivos Generados Automáticamente

El archivo también gestiona archivos generados por Visual Studio:

```
<Compile Update="Properties\Resources.Designer.cs">
```

```
<Compile Update="Properties\Settings.Designer.cs">
```

Funciones:

- Resources.Designer.cs
Maneja recursos como imágenes, textos e iconos.
- Settings.Designer.cs
Gestiona configuraciones del sistema (como cadenas de conexión).

Estos archivos se generan automáticamente y no deben modificarse manualmente.

34.5 Recursos del Proyecto

<EmbeddedResource Update="Properties\Resources.resx">

Este bloque indica que el archivo de recursos:

- Está incrustado dentro del ejecutable
- Contiene elementos visuales y configuraciones del sistema

34.6 Configuración de Settings

<None Update="Properties\Settings.settings">

Este archivo permite:

- Definir configuraciones del sistema
- Guardar parámetros como conexiones a base de datos
- Facilitar cambios sin modificar el código fuente

34.7 Importancia del Archivo en el Proyecto

El archivo .csproj es clave en la arquitectura del sistema porque:

- Centraliza la configuración del proyecto
- Permite gestionar dependencias fácilmente mediante NuGet
- Define cómo se construye y ejecuta la aplicación
- Facilita la portabilidad del proyecto a otros entornos

Conclusión

El archivo **Gestor de Horarios de Maestros.csproj** establece la base técnica del proyecto, definiendo tanto su estructura como sus dependencias. Gracias a su configuración, el sistema puede operar como una aplicación de escritorio basada en Windows Forms, con soporte para generación de reportes, acceso a múltiples bases de datos y manejo eficiente de configuraciones.

35. Archivo de Configuración de Usuario del Proyecto

Archivo: Gestor de Horarios de Maestros.csproj.user

El archivo .csproj.user es un archivo de configuración específico del usuario dentro de un proyecto .NET. Este archivo almacena preferencias locales del entorno de desarrollo y no forma parte esencial de la lógica del sistema.

35.1 Propósito del Archivo

El archivo .csproj.user tiene como objetivo:

- Guardar configuraciones personalizadas del usuario
- Mantener preferencias del entorno de desarrollo (Visual Studio)
- Almacenar rutas locales del sistema
- Registrar configuraciones de publicación

Este archivo es generado automáticamente por Visual Studio y puede variar entre diferentes equipos.

35.2 Configuración de Perfil de Publicación

```
<PropertyGroup>
```

```
  <_LastSelectedProfileId>D:\Folder\Programar\C#\Proyecto-Final---Proyecto-y-Gobierno-de-  
  TI\Gestor                                de                                Horarios                                de  
  Maestros\Properties\PublishProfiles\FolderProfile.pubxml</_LastSelectedProfileId>
```

```
</PropertyGroup>
```

Descripción:

- _LastSelectedProfileId almacena la ruta del último perfil de publicación utilizado.
- Este perfil define cómo se genera una versión ejecutable del sistema.
- En este caso, apunta a un perfil de publicación en carpeta (FolderProfile.pubxml).

Importancia:

- Permite recordar automáticamente la última configuración de despliegue utilizada.
 - Facilita el proceso de publicación del sistema.
-

35.3 Definición de Formularios del Proyecto

```
<ItemGroup>
```

```
  <Compile Update="FormAgregar.cs">
```

```
    <SubType>Form</SubType>
```

```
  </Compile>
```

```
...
```

</ItemGroup>

Este bloque define los archivos que Visual Studio reconoce como formularios (Windows Forms).

Formularios incluidos:

Archivo	Función dentro del sistema
ConfigConexion.cs	Configuración de la conexión a la base de datos
FormAgregar.cs	Registro de maestros y materias
FormAsignar.cs	Asignación de maestros a materias
FormBuscar.cs	Consulta de información
FormImprimir.cs	Generación de reportes PDF
FormModificar.cs	Edición de datos
Principal.cs	Formulario principal del sistema

35.4 Propiedad SubType = Form

<SubType>Form</SubType>

Esta propiedad indica que el archivo:

- Es un formulario visual
- Debe abrirse con el diseñador gráfico de Windows Forms
- Contiene interfaz de usuario

Esto permite que Visual Studio muestre correctamente el diseñador visual al abrir estos archivos.

35.5 Características del Archivo

El archivo .csproj.user presenta las siguientes características:

- Es específico del usuario (no del proyecto en sí)
 - Puede cambiar entre diferentes computadoras
 - No afecta la lógica ni funcionamiento del sistema
 - Generalmente no se incluye en repositorios (Git)
-

35.6 Relación con el Proyecto

Aunque este archivo no influye directamente en la ejecución del sistema, sí cumple un rol importante en el entorno de desarrollo:

- Mejora la experiencia del desarrollador
 - Facilita la gestión de formularios
 - Permite mantener configuraciones personalizadas
-

Conclusión

El archivo **Gestor de Horarios de Maestros.csproj.user** es un componente auxiliar del proyecto que almacena configuraciones locales del entorno de desarrollo. Aunque no afecta la ejecución del sistema, facilita la gestión de formularios y la publicación del proyecto, mejorando la experiencia de desarrollo en Visual Studio.

36. Archivo de Solución del Proyecto

Archivo: Gestor de Horarios de Maestros.slnx

El archivo .slnx representa la solución del proyecto en el entorno de desarrollo. Su función principal es agrupar y organizar uno o varios proyectos dentro de una misma estructura de trabajo.

36.1 Propósito del Archivo

El archivo de solución permite:

- Agrupar proyectos relacionados
- Facilitar la apertura del sistema en Visual Studio
- Definir qué proyectos forman parte de la solución
- Organizar el entorno de desarrollo

En este caso, la solución contiene un único proyecto.

36.2 Estructura del Archivo

```
<Solution>
```

```
<Project Path="Gestor de Horarios de Maestros.csproj" />
```

```
</Solution>
```

36.3 Descripción de los Elementos

Elemento	Descripción
----------	-------------

Solution	Elemento raíz que define la solución
----------	--------------------------------------

Elemento	Descripción
----------	-------------

Project	Indica el proyecto incluido en la solución
---------	--

Path	Ruta del archivo .csproj del proyecto
------	---------------------------------------

36.4 Proyecto Incluido

La solución incluye el siguiente proyecto:

- **Gestor de Horarios de Maestros.csproj**

Este proyecto contiene toda la lógica, interfaz y configuración del sistema.

36.5 Características del Archivo

- Utiliza un formato simplificado basado en XML
 - Es más ligero que el archivo tradicional .sln
 - Facilita la organización en proyectos pequeños o individuales
 - Permite una configuración clara y directa
-

36.6 Importancia dentro del Sistema

El archivo .slnx es importante porque:

- Permite abrir el proyecto completo con un solo archivo
 - Organiza la estructura general del sistema
 - Facilita la gestión del desarrollo en equipo
 - Actúa como punto de entrada para el entorno de desarrollo
-

36.7 Relación con Otros Archivos

Archivo	Función
---------	---------

.slnx	Define la solución
-------	--------------------

.csproj	Define el proyecto
---------	--------------------

Archivos .cs Contienen la lógica del sistema

Conclusión

El archivo **Gestor de Horarios de Maestros.slnx** define la solución del sistema, permitiendo organizar y cargar el proyecto dentro del entorno de desarrollo. Aunque su estructura es simple, cumple un papel fundamental como punto de acceso y organización del proyecto completo.

37. Archivo de Diseño del Formulario Principal

Archivo: Principal.Designer.cs

El archivo Principal.Designer.cs contiene la definición visual del formulario principal del sistema Gestor de Horarios de Maestros. Este formulario actúa como la interfaz principal desde la cual el usuario puede acceder a todas las funcionalidades del sistema.

Este archivo es generado automáticamente por Microsoft Visual Studio y forma parte parcial de la clase Principal.

37.1 Propósito del Formulario Principal

El formulario Principal tiene como objetivo:

- Servir como pantalla principal del sistema
 - Mostrar los datos de horarios
 - Permitir la navegación entre módulos
 - Facilitar la interacción del usuario con el sistema
-

37.2 Estructura General del Formulario

El formulario está compuesto por los siguientes elementos principales:

- Menú principal (menuStrip1)
 - Panel de filtros (panelFiltro)
 - Tabla de datos (dataGridView1)
 - Barra de control de ventana (menuStrip2)
-

37.3 Menú Principal (menuStrip1)

El menú principal contiene las opciones del sistema:

Opción	Función
 Agregar	Abre el formulario de registro de datos
 Asignar	Abre el formulario de asignación
 Modificar	Permite editar registros

Opción	Función
 Buscar	Permite consultar información
 Eliminar	Permite eliminar registros
 Conexión	Configuración de conexión a la base de datos
 Local	Configuración local
 Actualizar	Recarga los datos
 Imprimir	Genera reportes PDF

Cada opción está vinculada a un evento Click que ejecuta la funcionalidad correspondiente.

37.4 Panel de Filtro (panelFiltro)

Este panel permite filtrar la información mostrada en pantalla.

Controles incluidos:

Control	Función
label1	Texto “Maestro:”
comboBox1	Lista de maestros disponibles
btnBuscar	Ejecuta la búsqueda

Funcionamiento:

- El usuario selecciona un maestro
- Presiona el botón Buscar
- Se filtran los datos en la tabla

37.5 Tabla de Datos (dataGridView1)

El control DataGridView muestra los datos del sistema.

Características:

- Visualización de horarios
- Selección completa de filas
- Estilo personalizado (modo oscuro)
- Sin encabezados de fila
- Adaptable al tamaño del formulario

Configuración visual:

- Colores oscuros para interfaz moderna
- Encabezados resaltados
- Selección con color azul

Este control es el principal encargado de mostrar la información al usuario.

37.6 Barra de Control de Ventana (menuStrip2)

Este menú simula los controles de ventana personalizados:

Control	Función
✕ Cerrar	Cierra la aplicación
□ Maximizar	Maximiza la ventana
— Minimizar	Minimiza la ventana

Características:

- Interfaz personalizada (sin borde estándar)
 - Manejo manual de eventos de ventana
 - Permite arrastrar la ventana
-

37.7 Método InitializeComponent()

Este método es el núcleo del archivo y se encarga de:

- Crear todos los controles
- Configurar sus propiedades
- Definir posiciones y tamaños
- Asociar eventos

Se ejecuta automáticamente al iniciar el formulario.

37.8 Eventos Asociados

El formulario incluye múltiples eventos importantes:

Evento	Función
Principal_Load	Carga inicial del formulario

Evento	Función
agregarToolStripMenuItem_Click	Abrir formulario de agregar
asignarToolStripMenuItem_Click	Abrir formulario de asignar
modificarToolStripMenuItem_Click	Editar datos
buscarToolStripMenuItem_Click	Buscar registros
removeToolStripMenuItem_Click	Eliminar datos
conexiónToolStripMenuItem_Click	Configurar conexión
actualizarToolStripMenuItem_Click	Recargar datos
imprimirToolStripMenuItem_Click	Generar PDF
btnBuscar_Click	Ejecutar filtro
dataGridView1_CellContentClick	Interacción con tabla

37.9 Diseño Visual del Formulario

El formulario utiliza un diseño moderno con las siguientes características:

- Tema oscuro
- Colores personalizados (azul y gris)
- Íconos en los menús
- Interfaz sin bordes (FormBorderStyle.None)
- Ventana centrada al iniciar

37.10 Configuración General del Formulario

Propiedad	Valor
Tamaño	950 x 650
Color de fondo	Oscuro
Fuente	Segoe UI
Posición inicial	Centrado
Borde	Sin bordes

37.11 Relación con Otros Formularios

El formulario Principal actúa como punto de acceso a:

- FormAgregar
- FormAsignar
- FormModificar
- FormBuscar
- FormImprimir

Esto lo convierte en el centro de control del sistema.

Conclusión

El archivo Principal.Designer.cs define la interfaz gráfica principal del sistema, integrando navegación, visualización de datos y control de funcionalidades en un solo entorno. Su diseño moderno y organizado permite una interacción eficiente con el usuario y facilita el acceso a todos los módulos del sistema.

38. Lógica del Formulario Principal

Archivo: Principal.cs

El archivo Principal.cs contiene la lógica de funcionamiento del formulario principal del sistema Gestor de Horarios de Maestros. Este archivo controla la interacción con la base de datos, la gestión de eventos del sistema y la coordinación entre los diferentes módulos.

38.1 Propósito del Archivo

Este archivo se encarga de:

- Gestionar la conexión a la base de datos
 - Cargar y mostrar datos en la interfaz
 - Controlar eventos del sistema
 - Coordinar la navegación entre formularios
 - Implementar la lógica principal del sistema
-

38.2 Modo de Conexión (Local vs Remoto)

El sistema permite trabajar en dos modos:

- **Modo Remoto:** Conexión a base de datos MySQL
- **Modo Local:** Uso de base de datos SQLite

Variable utilizada:

```
private bool ModoLocal = false;
```

Funcionamiento:

- Si ModoLocal = true → usa SQLite
- Si ModoLocal = false → usa MySQL

Esto permite que el sistema sea portable y funcione sin conexión a internet.

38.3 Cadena de Conexión Dinámica

```
string connectionString
```

Esta propiedad:

- Selecciona automáticamente la cadena de conexión
- Lee desde app.config
- Usa:
 - "BaseLocal" para SQLite
 - "MiConexion" para MySQL

Incluye validación en caso de que la conexión no exista.

38.4 Creación de Conexiones

```
private IDbConnection CrearConexion()
```

Este método:

- Devuelve una conexión genérica (IDbConnection)
- Permite trabajar con diferentes motores de base de datos

Tipos utilizados:

- SQLiteConnection → modo local
 - MySqlConnection → modo remoto
-

38.5 Adaptadores de Datos

```
private DbDataAdapter CrearAdapter(string query, IDbConnection con)
```

Permite ejecutar consultas de forma genérica:

- SQLiteDataAdapter
- MySqlDataAdapter

Esto evita duplicar código y mejora la flexibilidad del sistema.

38.6 Inicialización del Sistema

Evento principal:

Principal_Load

Funciones:

- Inicializa la base de datos local
 - Recupera el modo de conexión guardado
 - Configura la interfaz
 - Carga datos en pantalla
-

38.7 Creación Automática de Base de Datos Local

Método:

InicializarBaseDeDatosLocal()

Este método:

- Crea automáticamente la base de datos SQLite
- Define las tablas:
 - Maestros
 - Materias
 - Cuatrimestres
- Crea la vista:
 - HorariosView

Esto permite que el sistema funcione sin instalación previa.

38.8 Carga de Datos en la Interfaz

CargarComboMaestros()

- Consulta la tabla Maestros
- Llena el ComboBox
- Agrega opción “Todos”

CargarGrid()

- Consulta la vista HorariosView

- Aplica filtros dinámicos
- Muestra datos en el DataGridView

Filtros disponibles:

- Maestro
- Sección
- Día
- Crédito
- Hora

38.9 Uso de Parámetros SQL

Método:

CrearParametro()

Ventajas:

- Evita inyección SQL
- Mejora seguridad
- Permite reutilización

38.10 Eventos del Sistema

Eventos principales:

Evento	Función
btnBuscar_Click	Filtra datos
comboBox1_SelectedIndexChanged	Actualiza resultados
dataGridView1_CellFormatting	Formatea la hora
Principal_Load	Inicializa sistema

38.11 Navegación entre Formularios

El sistema abre otros formularios mediante:

ShowDialog()

Formularios utilizados:

- FormAgregar

- FormAsignar
 - FormModificar
 - FormBuscar
 - FormImprimir
 - ConfigConexion
-

38.12 Eliminación de Registros

El sistema permite eliminar materias mediante:

- Selección en el DataGridView
- Confirmación del usuario
- Ejecución de consulta SQL:

DELETE FROM Materias WHERE IdMateria = @id

Incluye manejo de errores y actualización automática de la interfaz.

38.13 Generación de Reportes

Para generar los reportes Se utiliza la librería:

- **iTextSharp**

Permite:

- Crear archivos PDF
 - Exportar información del sistema
-

38.14 Manejo de Ventana Personalizada

Se implementa mediante:

[DllImport("user32.DLL")]

Funciones:

- Mover ventana sin bordes
 - Controlar minimizar, maximizar y cerrar
-

38.15 Cambio de Modo de Conexión

Evento:

localToolStripMenuItem_Click

Funciones:

- Cambia entre modo local y remoto
- Guarda preferencia del usuario
- Actualiza la interfaz

Esto mejora la experiencia del usuario y la persistencia del sistema.

38.16 Manejo de Errores

El sistema utiliza:

- Bloques try-catch
- Mensajes informativos (MessageBox)

Permite detectar errores como:

- Fallos de conexión
 - Problemas en consultas
 - Errores de ejecución
-

38.17 Buenas Prácticas Aplicadas

Uso de interfaces (IDbConnection)

Separación de lógica y diseño

Uso de parámetros SQL

Código reutilizable

Manejo de múltiples bases de datos

Persistencia de configuración

Creación automática de base de datos

Conclusión

El archivo Principal.cs representa el núcleo lógico del sistema, coordinando la interacción entre la interfaz, la base de datos y los distintos módulos. Su diseño flexible permite trabajar con múltiples motores de base de datos, implementar validaciones y ofrecer una experiencia de usuario completa y eficiente.

39. Archivo de Recursos del Formulario Principal

Archivo: Principal.resx

El archivo Principal.resx es un archivo de recursos asociado al formulario principal del sistema. Este archivo almacena información adicional utilizada por la interfaz gráfica, como configuraciones visuales, iconos y metadatos.

Forma parte de la estructura estándar de aplicaciones desarrolladas con Windows Forms en .NET.

39.1 Propósito del Archivo

El archivo .resx del modulo Principal se tuvo como finalidad implementar las siguientes del archivo asignado:

- Almacenar recursos del formulario
 - Separar datos visuales del código fuente
 - Facilitar la organización del proyecto
 - Permitir la reutilización de recursos
-

39.2 Formato del Archivo

El archivo está basado en **XML (Extensible Markup Language)** y sigue el estándar **Microsoft ResX Schema**.

Incluye elementos como:

- resheader → Información general del archivo
 - data → Recursos almacenados
 - metadata → Configuración adicional
 - assembly → Referencias a bibliotecas
-

39.3 Encabezados del Archivo

```
<resheader name="resmimetype">  
  <value>text/microsoft-resx</value>  
</resheader>
```

Función:

- Define el tipo de archivo
- Indica que es un archivo de recursos de Microsoft

Otros encabezados incluyen:

- Versión del formato
 - Lector (reader)
 - Escritor (writer)
-

39.4 Metadatos del Formulario

```
<metadata name="menuStrip1.TrayLocation">
```

```
<value>17, 17</value>
```

```
</metadata>
```

Descripción:

- Guarda la posición de componentes dentro del diseñador
- No afecta directamente la ejecución del sistema
- Es utilizado por Visual Studio para mantener el diseño

Ejemplos:

Elemento	Función
menuStrip1.TrayLocation	Posición del menú principal
menuStrip2.TrayLocation	Posición del menú secundario

39.5 Recursos Gráficos

```
<data name="$this.Icon" type="System.Drawing.Icon">
```

Este elemento almacena:

- El icono del formulario principal
- Codificado en formato Base64

Función:

- Mostrar el icono en la ventana
 - Representar visualmente la aplicación
-

39.6 Referencias de Ensamblados

```
<assembly alias="System.Drawing" name="System.Drawing, Version=4.0.0.0">
```

Función:

- Referenciar bibliotecas necesarias
 - Permitir el uso de tipos gráficos como:
 - Iconos
 - Colores
 - Imágenes
-

39.7 Tipos de Recursos Soportados

El Por medio de este archivo, las funciones .resx para almacenar fueron:

- Textos
- Imágenes
- Iconos
- Archivos binarios
- Configuraciones

En este proyecto se utiliza principalmente para:

- Icono del sistema
- Configuración del diseñador

39.8 Relación con Otros Archivos

El archivo Principal.resx trabaja junto con:

Archivo	Función
Principal.cs	Lógica del formulario
Principal.Designer.cs	Diseño visual
Principal.resx	Recursos

39.9 Características Importantes

- Se genera automáticamente por Visual Studio
- No requiere edición manual
- Permite internacionalización (multi-idioma)
- Facilita mantenimiento del sistema

Conclusión

El archivo Principal.resx permite gestionar los recursos visuales y metadatos del formulario principal, separando estos elementos del código fuente. Esto mejora la organización, facilita el mantenimiento del sistema y permite una mejor gestión de la interfaz gráfica dentro del entorno de desarrollo.

40. Archivo de Inicio del Sistema

Archivo: Program.cs

40.1 Propósito del Archivo

El archivo Program.cs es el punto de entrada principal del sistema. Es el encargado de iniciar la ejecución de la aplicación y abrir el formulario principal.

40.2 Método Principal (Main)

```
static void Main()
{
    ApplicationConfiguration.Initialize();
    Application.Run(new Principal());
}
```

40.3 Descripción del Funcionamiento

Instrucción	Función
ApplicationConfiguration.Initialize()	Inicializa la configuración de la aplicación (estilos visuales, DPI, etc.)
Application.Run(new Principal())	Inicia la aplicación mostrando el formulario principal

40.4 Atributo STAThread

[STAThread]

Este atributo indica que:

- La aplicación utiliza un modelo de subprocesos de un solo hilo
 - Es necesario para componentes gráficos (Windows Forms)
 - Permite el correcto funcionamiento de controles visuales
-

40.5 Importancia

Este archivo es fundamental porque:

- Marca el inicio del sistema
- Controla la ejecución de la aplicación

- Define qué formulario se mostrará al usuario

Conclusión

El archivo Program.cs es el punto de partida del sistema, encargado de inicializar la aplicación y ejecutar el formulario principal, permitiendo que el usuario interactúe con el sistema desde el inicio.

41. Módulo de Validación de Horarios

Archivo: ValidadorHorario.cs

41.1 Propósito del Archivo

Este archivo implementa la lógica para validar conflictos de horarios en el sistema. Su objetivo es evitar que un maestro tenga clases asignadas en el mismo horario.

41.2 Clase HorarioSimple

```
public class HorarioSimple
```

Esta clase representa un horario básico con los siguientes atributos:

Atributo	Tipo	Descripción
----------	------	-------------

Maestro	string	Nombre del maestro
---------	--------	--------------------

Dia	string	Día de la clase
-----	--------	-----------------

HoraInicio	int	Hora de inicio
------------	-----	----------------

HoraFin	int	Hora de finalización
---------	-----	----------------------

41.3 Clase ValidadorHorarios

```
public class ValidadorHorarios
```

Esta clase contiene la lógica de validación de horarios.

41.4 Método Principal de Validación

```
public static bool HayChoque(HorarioSimple nuevo, List<HorarioSimple> existentes, out string mensaje)
```

41.5 Parámetros del Método

Parámetro Descripción

nuevo	Horario que se desea agregar
existentes	Lista de horarios ya registrados
mensaje	Mensaje de error en caso de conflicto

41.6 Lógica de Validación

El método realiza:

1. Recorre todos los horarios existentes
2. Verifica si coinciden:
 - El mismo maestro
 - El mismo día
3. Evalúa si hay cruce de horas:

nuevo.HoraInicio < h.HoraFin && nuevo.HoraFin > h.HoraInicio

41.7 Resultado del Método

Resultado Significado

true	Existe conflicto de horario
false	No hay conflicto

41.8 Mensaje de Error

Si hay conflicto, devuelve un mensaje como:

El maestro Juan Pérez ya tiene clase el lunes de 8:00 a 10:00.

41.9 Importancia del Módulo

Este módulo es clave porque:

- Evita errores en la asignación de clases
 - Garantiza consistencia en los horarios
 - Mejora la integridad del sistema
 - Previene duplicidad de horarios
-

41.10 Ventajas de la Implementación

Código reutilizable
Validación centralizada
Fácil integración con formularios
Mejora la calidad del sistema

Conclusión

El archivo ValidadorHorario.cs proporciona una solución eficiente para detectar conflictos de horarios, asegurando que los maestros no tengan asignaciones superpuestas. Esto fortalece la lógica del sistema y garantiza una correcta gestión académica.

42. Archivo de Configuración de la Aplicación

Archivo: app.config

42.1 Propósito del Archivo

El archivo app.config es el encargado de almacenar la configuración general del sistema. Permite definir parámetros importantes sin necesidad de modificar el código fuente.

42.2 Funciones Principales

Este archivo permite:

- Definir cadenas de conexión a bases de datos
 - Almacenar configuraciones del sistema
 - Gestionar preferencias del usuario
 - Facilitar cambios sin recompilar la aplicación
-

42.3 Sección de Configuración (configSections)

<configSections>

Esta sección define grupos de configuración personalizados.

En este caso:

- Se crea un grupo llamado userSettings
 - Permite guardar configuraciones del usuario
 - Se vincula con la clase Properties.Settings del proyecto
-

42.4 Cadenas de Conexión (connectionStrings)

<connectionStrings>

Aquí se definen las conexiones a bases de datos.

42.5 Conexión Remota (MySQL)

<add name="BaseRemota"

connectionString="Server=IP_DE_TU_PI; Database=GestorHorarios; Uid=admin_user;
Pwd=tu_password;"

providerName="MySql.Data.MySqlClient" />

Características:

- Conecta a un servidor remoto
 - Usa MySQL
 - Requiere:
 - Dirección IP
 - Usuario
 - Contraseña
-

42.6 Conexión Local

<add name="BaseLocal"

connectionString="server=localhost;database=GestorHorarios;uid=root;pwd=root;" />

Características:

- Base de datos local
 - Útil para pruebas o uso sin internet
 - Permite trabajar en modo offline
-

42.7 Configuración de Usuario (userSettings)

<userSettings>

Permite guardar preferencias del sistema.

42.8 Configuración del Modo de Conexión

<setting name="ModoConexion" serializeAs="String">

<value>Remoto</value>

</setting>

Función:

- Guarda el modo actual del sistema:
 - Remoto → MySQL
 - Local → SQLite

Importancia:

- Permite recordar la preferencia del usuario
 - Evita tener que configurar cada vez que se abre el sistema
-

42.9 Relación con el Código

Este archivo es utilizado directamente en:

- Principal.cs → para obtener la conexión
- Properties.Settings → para guardar preferencias

Ejemplo:

```
ConfigurationManager.ConnectionStrings["BaseLocal"]
```

42.10 Ventajas del Uso de app.config

Separación de configuración y código

Mayor seguridad (no hardcodear credenciales)

Fácil mantenimiento

Flexibilidad en despliegue

Adaptabilidad a distintos entornos

42.11 Buenas Prácticas

- No exponer contraseñas en producción
 - Usar variables seguras o cifrado
 - Mantener nombres de conexiones claros
 - Validar siempre las configuraciones
-

42.12 Importancia en el Proyecto

Este archivo es clave porque:

- Permite cambiar entre modo local y remoto

- Define cómo el sistema se conecta a la base de datos
 - Guarda configuraciones persistentes
 - Facilita el funcionamiento completo del sistema
-

Conclusión

El archivo `app.config` es un componente fundamental del sistema, ya que centraliza la configuración de conexiones y preferencias del usuario. Su correcta implementación permite que el sistema sea flexible, seguro y adaptable a diferentes entornos de ejecución.
